

Let's build GPT： from scratch, in code, spelled out

从零构建 GPT：代码逐行详解（字幕版）｜亲手实现 Transformer 语言模型

时长 1:56:19 | 句数 1157 | 术语 51 | 章节 16

摘要

本讲共 16 章，从「大家好,你们可能已经听说过聊」到「训练数据 · 微调」。全文 1157 句、1156 句已译成中文，标注了 53 个 AI 术语（主要集中在神经网络基础、数学与微积分、训练与优化）。

16 chapters, 1157 sentences (1156 translated), 53 AI terms annotated. Built fully offline with opus-mt + local glossary.

大家好,你们可能已经听说过聊 00:00

00:00 Hi everyone. So by now you have probably heard of chatGPT. It has taken the world and AI community
大家好,你们可能已经听说过聊天了

00:05 by storm and it is a system that allows you to interact with an AI and give it text-based tasks.
并赋予它基于文本的任务。

00:12 So for example, we can ask chatGPT to write us a small haiku about how important it is that people
比如,我们可以请聊天GPT给我们写一个小的口号, 说人们有多重要

00:17 understand AI and then they can use it to improve the world and make it more prosperous. So when
了解AI,然后他们就可以利用它改善世界,使之更加繁荣。

00:21 we run this, AI knowledge brings prosperity for all to see embrace its power. Okay, not bad.
人工智能知识为大家带来繁荣 让人们看到拥抱它的力量

00:29 And so you could see that chatGPT went from left to right and generated all these words
让你可以看到聊天GPT 从左向右 产生出所有这些词

00:33 seek sort of sequentially. Now I asked it already the exact same prompt a little bit earlier
相顺序查找。 现在我更早地询问了相同的提示

00:39 and it generated a slightly different outcome. AI's power to grow, ignorance holds us back,
并且产生了一个略有不同的结果。AI的力量成长,无知让我们退缩,

00:44 learn, prosperity waits. So pretty good in both cases and slightly different. So you can see that
学习,繁荣等待,在两种情况中都很好

00:50 chatGPT is a probabilistic system and for any one prompt it can give us multiple answers
COTPGPT是一个概率系统,对于任何一个能迅速给我们多个答案的系统来说,它都可以提供多重答案

00:55 sort of replying to it. Now this is just one example of the prompt. People have come up with
也即是应答。现在这只是一个急事的例子。人们已经想到了

01:01 many, many examples and there are entire websites that index interactions with chatGPT.
有许多例子和整个网站都与聊天点和点对点进行索引化互动。

01:06 And so many of them are quite humorous. Explain HTML to me like I'm a dog,
说得像狗一样给我解释一下HTML

01:12 write release notes for chess too, write a note about Elon Musk buying a Twitter and so on.
写下象棋的发布笔记,

机器翻译 · 神经网络 01:19

01:19 So as an example, please write a breaking news article about a leaf falling from a tree
举例来说,请写一篇关于树叶落下的文章

01:24 and a shocking turn of events. A leaf has fallen from a tree in the local park.
一片叶子从当地公园的树上掉下来

01:28 Witnesses report that the leaf which was previously attached to a branch of a tree
目击者报告说,以前树枝上附着的叶叶子

01:32 detached itself and fell to the ground. Very dramatic. So you can see that this is a
自己脱落,摔倒在地上。非常戏剧性。所以你可以看到这是

01:37 pretty remarkable system and it is what we call a language model because it models the
相当惊人的系统, 这就是我们所谓的语言模式, 因为它模拟

01:43 sequence of words or characters or tokens more generally and it knows how sort of words
更笼统的文字、字符或符号序列,它知道如何排序单词

01:49 follow each other in English language. And so from its perspective what it is doing is it is
以英语互相跟随。所以从它的角度看,它在做什么是

01:55 completing the sequence. So I give it the start of a sequence and it completes the sequence with
完成序列。 所以我给它一个序列的开始, 它以

02:01 the outcome and so it's a language model in that sense. Now I would like to focus on the under
结果是,所以它是一个语言模式 在这个意义上。现在我想集中谈谈

02:06 the hood of under the hood components of what makes chatGPT work. So what is the neural network
是什么让聊天GPT起作用的。

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

02:12 under the hood that models the sequence of these words? And that comes from this
以这个罩下 模拟这些词的顺序?

02:18 paper called attention is all you need. In 2017 a landmark paper in AI that produced and
2017年,在大赦国际上发表的一份里程碑式论文,

attention
注意力 注意力
神经网络基础

02:25 proposed the transformer architecture. So GPT is short for generatively pre-trained transformer.
提议了变压器结构。 所以GPT对于基因化的预训练变压器来说是短的。

transformer [træns'fɔ:mə]
Transformer 中文为“变换器”，但不够达雅，日常还是称英文 Transformer 中文为“变换器”，但不够达雅，日常还是称英文
神经网络基础

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

02:34 So transformer is the neural net that actually does all the heavy lifting under the hood.
所以变压器就是神经网 能够真正完成引擎盖下的所有重力提升

transformer [træns'fɔ:mə]
Transformer 中文为“变换器”，但不够达雅，日常还是称英文 Transformer 中文为“变换器”，但不够达雅，日常还是称英文
神经网络基础

neural net
神经网络 神经网络
神经网络基础

02:38 It comes from this paper in 2017. Now if you read this paper, this reads like a pretty
2017年的报纸上写着,如果你读了这本书,

02:44 random machine translation paper and that's because I think the authors didn't fully
机器翻译纸,因为我认为作者们没有完全

machine translation
机器翻译 The task of automatically transforming text (or speech) from one natural language into another.
神经网络基础

02:47 anticipate the impact that the transformer would have on the field and this architecture that they
变压器对实地和这一结构的预期影响

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

02:52 produced in the context of machine translation in their case actually ended up taking over the
以机器翻译方式制作的

machine translation
机器翻译 The task of automatically transforming text (or speech) from one natural language into another.
神经网络基础

02:57 rest of AI in the next five years after. And so this architecture with minor changes was
之后五年内,AI的其余部分将在未来5年。因此这个结构有了小的变化

03:03 copy pasted into a huge amount of applications in AI in more recent years and that includes
近年来,在AI类中复制成为大量应用,包括:

03:10 at the core of chatGPT. Now we are not going to what I'd like to do now is I'd like to
现在我不想做我想做的事了

03:15 build out something like chatGPT but we're not going to be able to of course reproduce chatGPT.
建立像聊天GPT这样的东西 但我们当然不能复制聊天GPPT

03:21 This is a very serious production grade system. It is trained on a good chunk of internet and
这是一个非常严重的生产等级制度,它接受大量互联网和因特网培训。

chunk [tʃʌŋk]
词块 词块
神经网络基础

03:28 then there's a lot of pre- training and fine-tuning stages to it and so it's very complicated.
然后有很多训练前和微调阶段 都非常复杂

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

03:33 What I'd like to focus on is just to train a transformer based language model and in our
我想集中关注的是 训练一个基于变压器的语言模型

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

03:38 case it's going to be a character level language model. I still think that is a very
情况是,这将是个性级语言模式。

03:43 educational with respect to how these systems work. So I don't want to train on the chunk of
教育如何运用这些系统。所以我不想训练

chunk [tʃʌŋk]
词块 词块
神经网络基础

03:48 internet we need a smaller dataset. In this case I propose that we work with my favorite toy dataset
我们需要一个较小的互联网数据集。 在这种情况下,我建议我们用我最喜欢的玩具数据集

03:54 it's called Tiny Shakespeare and what it is is basically it's a concatenation of all of the
基本上它是一个所有

03:59 works of Shakespeare in my understanding and so this is all of Shakespeare in a single file.
莎士比亚的作品 在我的理解, 所以这是所有的莎士比亞 在单一的文件。

04:05 This file is about one megabyte and it's just all of Shakespeare and what we are going to
这个文件是关于一个兆字节的 和它只是所有的莎士比亚 我们将要要做什么

04:10 do now is we're going to basically model how these characters follow each other. So for example
现在,我们基本上要模拟这些字符 如何互相跟随。例如

04:15 given a chunk of these characters like this given some context of characters in the past
给给定的这些字符块中的一块, 像这样的字符块, 给给给定过去一些字符的背景

chunk [tʃʌŋk]
词块 词块
神经网络基础

04:22 the transformer neural network will look at the characters that I've highlighted
变压器神经网络将查看 我突出显示的字符

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

04:25 and it's going to predict that G is likely to come next in the sequence and it's going to do
并且它预测G很可能在序列中排在下一个 然后它会做

04:30 that because we're going to train that transformer on Shakespeare and it's just going to try to
因为我们要去莎士比亚训练变压器 它只是试着

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

04:35 produce character sequences that look like this and in that process is going to model all the
生成字符序列, 产生类似这样的字符序列, 在此过程中将模拟所有

04:41 patterns inside this data. So once we've trained the system I just like to give you a preview
数据中的模式模式。所以一旦我们训练了系统,我只想给你们一个预览

04:46 we can generate infinite Shakespeare and of course it's a fake thing that looks kind of like
我们可以产生无限的莎士比亚 当然,这是假的东西 看起来有点像

04:52 Shakespeare apologies for there's some jank that I'm not able to resolve in in here but
莎士比亚为一些我无法解决的弹痕道歉 但是

05:03 you can see how this is going character by character and it's kind of like predicting
可以看到这如何是 性格的运行字符 字符和它有点像预测

05:08 Shakespeare-like language. So verily my lord the sites have left the again the king coming with my
莎士比亚语言一样的语言。所以,我的大人,这些网站确实离开了国王,他带着我的

05:15 curses with precious pale and then Traniosa something else etc and this is just coming out of the
以珍贵的苍白诅咒 然后特拉尼奥萨 还有其他的东西等等, 而这只是从

05:22 transformer in a very similar manner as it would come out in chat gpt in our case character by
以非常相似的方式 以聊天时出现的变压器 。

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

05:27 character in chat gpt it's coming out on the token by token level and tokens are these a
符号是这些

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

05:33 sort of like little subword pieces so they're not word level they're kind of like word chunk level
类似小子字节的子字块,所以它们不是文字级 它们有点像文字块级

chunk [tʃʌŋk]
词块 词块
神经网络基础

数据集 · 神经网络 05:40

05:40 and now I've already written this entire code to train these transformers and it is in a
现在我已经写了整个代码 来训练这些变压器 它在

05:48 GitHub repository that you can find and it's called nano gpt so nano gpt is a repository that
吉特Hub 库,你可以找到它 它被称为纳米Gpt 所以纳米Gpts是一个存储器

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

05:54 you can find on my github and it's a repository for training transformers on any given text
你可以在我的 Guthub 上找到它, 它是一个存储库, 用于培训任何文本的变压器

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

06:01 and what I think is interesting about it because there's many ways to train transformers
我认为这很有趣 因为有很多方法可以训练变压器

06:04 about this is a very simple implementation so it's just two files of 300 lines of code each
有关这个的是一个非常简单的执行, 所以它只是两个文件, 每300行代码

06:10 one file defines the gpt model the transformer and one file trains it on some given text data set
1 文件定义了变压器的 gpt 模型, 还有一个文件在某个特定文本数据集上对它进行训练

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

06:16 and here I'm showing that if you train it on a open web text data set which is a fairly
我在这里展示 如果你用开放的网络文本数据来训练它 这是一种公平的

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ，用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

06:20 large data set of web pages then I reproduce the the performance of gpt 2 so gpt 2 is an early
网页的大型数据集, 然后我就复制 gpt2 的性能, 所以 gppt2 是早期

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ，用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

06:27 version of open ai's gpt from 2017 if I recall correctly and I've only so far reproduced the
2017年开放版的 Ai 空格,如果我记起来正确的话,我至今只复制了

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

06:35 the smallest 124 million parameter model but basically this is just proving that the code
最小的1.24亿参数模型 但基本上这证明代码

parameter [pə'ræmitə]
参数 机器学习系统自行训练的模型的变量。例如，权重 就是一种参数，它们的值是机器 学习系统通过连续的训练迭代逐渐学习到的。与超参数 相对。
神经网络基础

06:39 base is correctly arranged and I'm able to load the neural network weights that open ai has released
基地设置正确,我能够装载 开放的电源释放的神经网络重量

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0，则相应的特征对模型来说没有任何贡献。
神经网络基础

06:45 later so you can take a look at the finished code here in nano gpt but what I would like
稍后你就可以看看这里的纳米格普特的完成代码 但我想看什么

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

06:51 to do in this lecture is I would like to basically write this repository from scratch
在这次演讲中,我想写这个库 基本上是为了从零开始

06:57 so we're going to begin with an empty file and we're going to define a transformer piece by piece
所以我们要从空文件开始 逐个定义变压器

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

07:03 we're going to train it on the tiny shakespeare data set and we'll see how we can then
我们用小的摇沙子数据集 来训练它 然后看看我们怎样才能

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API，用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

07:08 generate infinite shakespeare and of course this can copy paste to any arbitrary text data set
生成无限的电抖平, 当然可以复制到任意文本数据集

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ，用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

07:13 that you like but my goal really here is to just make you understand and appreciate
让你理解和欣赏

07:18 how under the hood chat gpt works and really all that's required is a proficiency in python
兜帽会下面的聊天如何运作 真正需要的就是精通俾顿

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

07:26 and some basic understanding of calculus and statistics and it would help if you also see
以及一些对微积分和统计的基本了解,如果你们能看见

07:31 my previous videos on the same youtube channel in particular my make more series where I define
我之前在同一Youtube频道上的视频 特别是我制作的更多系列

07:39 smaller and simpler neural network language models so multi-alial perceptrons and so on
小型和简易神经网络语言模型,如多向感应器等

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

07:44 it really introduces the language modeling framework and then here in this video we're
真正引入语言建模框架 然后在这个视频里我们

07:49 going to focus on the transformer neural network itself okay so I created a new google collab
将关注变压器神经网络本身 所以我创造了一个新的谷歌库

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

07:55 jupyret notebook here and this will allow me to later easily share this code that we're going to
Jupyret笔记本在这里,这让我稍后可以轻松分享 这个代码,我们将要使用

08:00 develop together with you so you can follow along so this will be in a video description later
与你们一起开发,以便你们可以跟随而成,这样以后将在录像中加以描述。

08:05 now here I've just done some preliminaries I downloaded the data set the tiny shakespeare
刚刚我刚做了一些初步的准备 我下载了数据组

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ， 用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

08:10 data set at this URL and you can see that it's about a one megabyte file then here I open the
URL 的数据集, 您可以看到它是一个兆字节文件, 然后在这里打开

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ， 用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

08:15 input.txt file and just read in all the text at the string and we see that we are working with
输入. txt 文件,然后在字符串的所有文本中读取,我们看到我们正与

08:20 one million characters roughly and the first 1000 characters if we just print them out are basically
大约100万个字符, 以及头1000个字符 如果我们只要打印出来

08:26 what you would expect this is the first 1000 characters of the tiny shakespeare data set roughly up to
也就是你所期望的 这是最初1000个字 摇摇树类小数据组中

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ， 用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

08:32 here so so far so good next we're going to take this text and the text is a sequence of characters
下个好结果,我们接下来要拍这个文本 文本是字符序列

08:39 in python so when I call the set constructor on it I'm just going to get the set of all the
皮松的Python 所以当我打电话叫 设置的建筑工人在上面时 我就会得到

08:45 characters that occur in this text and then I call list on that to create a list of those
字符中的字符,此文本中出现字符,然后我在此时拨入列表,以创建这些字符的列表

08:51 characters instead of just a set so that I have an ordering an arbitrary ordering and then I sort that
而不是一个字符,而不是仅仅一组, 所以我有命令 一个任意的命令 然后我排序

08:58 so basically we get just all the characters that occur in the entire data set and they're
所以基本上我们得到所有字符 在整个数据集中出现的所有字符

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ， 用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

09:01 sorted now the number of them is going to be our vocabulary size these are the possible
已经对数量进行了分类, 数目将是我们的词汇大小。 这些是可能的

09:06 elements of our sequences and we see that when I print here the characters there's 65 of them
我们的序列元素元素 我们看到当我在这里打印这些字符时, 有65个

09:13 in total there's a space character and then all kinds of special characters and then uh capitals
总共有一个空间字符 然后各种特殊字符 然后是首都

09:19 and lowercase letters so that's our vocabulary and that's the sort of like possible uh characters
和小写字母,所以这就是我们的词汇表 这就是可能的人

09:25 that the model can see or emit okay so next we would like to develop some strategy to tokenize
模型能看见或发出 都好,所以接下来我们想制定一些战略

09:32 the input text now when people say tokenize they mean convert the raw text as a string
当人们说表示符号时, 输入文本现在的输入文本, 意思是将原始文本转换成字符串

09:38 to some sequence of integers according to some note according to some vocabulary of possible
根据某些可能的词汇表根据一些注释的

09:44 elements so as an example here we are going to be building a character level language model so
元素,作为一个例子 我们将要建立一个 性格级别语言模式

09:49 we're simply going to be translating individual characters into integers so let me show you
我们只是将个人字符转换成整数,让我给你们看看

09:54 a chunk of code that sort of does that for us so we're building both the encoder and the decoder
大量的代码,可以为我们做这些 所以我们正在建造编码器和解码器

chunk [tʃʌŋk]
词块 词块
神经网络基础

10:00 and let me just talk through what's happening here when we encode an arbitrary text like hi
让我来谈谈这里发生的事情 当我们把一个任意的文本编码成喜

10:05 there we're going to receive a list of integers that represents that string so for example 46 47
在那里,我们将收到一份整数清单 代表这个字符串的整数,例如46 47

10:13 and etc and then we also have the reverse mapping so we can take this list and decode it to get back
等,然后我们还有反向映射图 这样我们可以拿下这个列表 并解码它回来

10:20 the exact same string so it's really just like a translation to integers and back for arbitrary
完全相同的字符串,所以它就像一个翻译 转换成整数和反向任意

10:25 string and for us it is done on a character level now the way this was achieved is we just iterate
对我们来说,字符串是在字符级别完成的 现在实现的方式是 我们只是转动

10:32 over all the characters here and create a lookup table from the character to the integer and vice
在此所有字符上方的字符, 并创建一个从字符到整数和反差的搜索表格

10:37 versa and then to encode some string we simply translate all the characters individually and to
然后将一些字符串编码, 我们简单地将所有字符逐个翻译

10:42 decode it back we use the reverse mapping and concatenate all of it now this is only one of
我们用反向绘图 将全部连接起来 现在这只是其中之一

10:49 many possible encodings or many possible sort of tokenizers and it's a very simple one but there's
有很多可能的编码或许多可能的信标 和很多可能的代用符号, 它是一个非常简单的

10:54 many other schemas that people have come up with in practice so for example google uses a sentence
人们在实践中也想出许多其他的 方法,例如谷歌使用一个句子

10:59 piece so sentence piece will also encode text into integers but in a different schema and using a
将文字编码成整数,但以不同的公式和

11:07 different vocabulary and sentence piece is a subword sort of tokenizer and what that means is that
不同的词汇和句子是象征符号的子词,这意味着

11:15 you're not encoding entire words but you're not also encoding individual characters it's it's a
你没有把全部字词编码起来 但你也没有同时将个人字符编码起来

11:19 subword unit level and that's usually what's adopted in practice for example also openai
子词组单位级别, 而这是通常实践中采用的方法, 例如开胃

11:25 has this library called tick token that uses a byte pair encoding tokenizer and that's what GPT uses
这个库有一个名为“ 滴答器” 的库, 它使用对对字节编码符号, GPT 就是这样使用的

> **gpt**
> 生成式预训练Transformer模型 生成式预训练Transformer模型
> 神经网络基础

11:33 and you can also just encode words into like hell world into lists of integers so as an example
也可以将文字编码成地狱世界, 以整数列表作为例子

11:39 i'm using the tick token library here i'm getting the encoding for GPT 2 or that was used for GPT 2
我正在使用这里的滴答器库 我正获取 GPT2 或GPT2的编码

> **gpt**
> 生成式预训练Transformer模型 生成式预训练Transformer模型
> 神经网络基础

11:46 instead of just having 65 possible characters or tokens they have 50 000 tokens and so when
而不是仅仅有65个可能的字符或符号,他们有50 000个象征性的,所以当

11:53 they encode the exact same string hi there we only get a list of three integers but those integers are
编码了完全相同的字符串 在那里我们只得到一个三个整数列表 但是那些整数

11:59 not between 0 and 64 they are between 0 and 50 256 so basically you can trade off the codebook size
介于0到64之间,不介于 0到50之间, 256, 所以基本上你可以交换代码簿大小

12:10 and the sequence lengths so you can have very long sequences of integers with very small
和序列长度,这样您就可以拥有非常长的整数序列,且极小

12:15 vocabularies or you can have short sequences of integers with very large vocabularies and so
词汇表或者你可以有短序数的整数序列, 并且有非常大的词汇表等等

12:24 typically people use in practice these subword encodings but i'd like to keep our tokenizer
通常人们在实践时使用这些子字编码, 但我想保留我们的代用符咒

12:29 very simple so we're using character level tokenizer and that means that we have very small
很简单 所以我们使用品格级符号仪 这意味着我们非常小

12:34 codebooks we have very simple encode and decode functions but we do get very long sequences
代码簿 我们有非常简单的编码和解码功能 但我们得到的序列很长

12:40 as a result but that's the level at which we're going to stick with this lecture because it's
但它是我们继续这个演讲的层次 因为

12:44 the simplest thing okay so now that we have an encoder and a decoder effectively a tokenizer
最简单的事情是好的 所以现在我们有了编码器 和一个解码器 一个有效的代碼器

12:50 we can tokenize the entire training set of Shakespeare so here's a chunk of code that does that
我们可以象征整个莎士比亚的训练 所以这里有一大块代码 这样做

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

chunk [tʃʌŋk]
词块 词块
神经网络基础

12:55 and i'm going to start to use the pytorch library and specifically the torch. tensor from the
我将开始使用 火炬图书馆, 特别是火炬. 电离器

pytorch
PyTorch PyTorch
代码与工程

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

12:59 pytorch library so we're going to take all of the text in tiny Shakespeare encode it and then
皮口切图书馆 所以我们要把所有的文本 都用小莎士比亚编码 然后

pytorch
PyTorch PyTorch
代码与工程

13:05 wrap it into a torch. tensor to get the data tensor so here's what the data tensor looks like
把它包成火炬 电压器来获取数据

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

13:10 when i look at just the first 1000 characters or the 1000 elements of it so we see that we have a
当我只看第一个1000个字符或它中的1000个元素时, 我们可以看到我们有一个

13:16 massive sequence of integers and this sequence of integers here is basically an identical translation
整数的宏序列和此整数序列基本上是一个相同的翻译

13:22 of the first 1000 characters here so i believe for example that zero is a new line character
首1000个字符中的第一个字符,所以我认为,例如,零是一个新线字符

13:28 and maybe one is a space not a hundred percent sure but from now on the entire data set
也许一个空间不是100%确定, 但从现在开始整个数据集上

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ， 用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

13:33 of text is re-represented as just it's just stretched out as a single very large
文本的重新表示,因为它只是作为一个非常大的单体延伸出来

13:38 sequence of integers let me do one more thing before we move on here i'd like to separate out
整数序列让我做一件事 在我们继续之前 我想分开

训练集 · 损失函数 13:43

13:43 our data set into a train and a validation split so in particular we're going to take the first
我们的数据组 进入火车和验证分解, 所以我们将采取第一个

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ， 用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

validation
验证 验证
训练与优化

13:49 90 percent of the data set and consider that to be the training data for the transformer
90%的数据集,并视之为变压器的培训数据

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API，用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

13:54 and we're going to withhold the last 10 percent at the end of it to be the validation data
最终我们保留最后10%的10% 作为验证数据

validation
验证 验证
训练与优化

13:59 and this will help us understand to what extent our model is overfitting so we're going to
这将有助于我们了解我们的模型 有多适合,所以我们要

overfitting
过拟合 过拟合
训练与优化

14:03 basically hide and keep the validation data on the side because we don't want just a perfect
基本上隐藏并保存验证数据 在侧面,因为我们不希望仅仅一个完美的

validation
验证 验证
训练与优化

14:07 memorization of this exact Shakespeare we want a neural network that sort of creates Shakespeare-like
莎士比亚的记忆 我们想要一个神经网络 来创造像莎士比阿一样的

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

14:13 text and so it should be fairly likely for it to produce the actual like stowed away
文字,所以它应该很有可能产生 像积载的一样的实际

14:21 true Shakespeare text and so we're going to use this to get a sense of the overfitting
莎士比亚的正文,所以我们用这个来 获得一种感知

overfitting
过拟合 过拟合
训练与优化

14:27 okay so now we would like to start plugging these text sequences or integer sequences into
开始插入这些文本序列或整数序列

14:32 the transformer so that it can train and learn those patterns now the important thing to realize
变压器,以便它能训练并学习这些模式 现在重要的是要实现

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

14:37 is we're never going to actually feed entire text into transformer all at once that would be
我们永远不会真的 将整个文本 输入变压器

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

14:42 computationally very expressive and prohibitive so when we actually train a transformer on a lot
计算非常清晰,令人望而却步 所以当我们训练变压器时

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

14:47 of these data sets we only work with chunks of the data set and when we train the transformer
在这些数据集中,我们只处理数据集的块数和当我们训练变压器时

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ，用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

14:51 we basically sample random little chunks out of the training set and train on just chunks at a
我们基本上从训练中抽取 随机的少量小块

sample
样本 样本
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

14:56 time and these chunks have basically some kind of a length and some maximum length now the maximum
这些块块的长度基本上 具有某种长度, 并且有某些最大长度 现在最大

15:04 length typically at least in the code I usually write is called block size you can you can find
通常至少在我的代码中,我通常写的代码的长度通常被称为块大小,您可以找到

15:10 it on the different names like context length or something like that let's start with the block
以块开头

15:14 size of just eight and let me look at the first train data characters the first block size
仅8个大小,让我看看第一个列车数据字符

15:19 plus one characters I'll explain why plus one in a second so this is the first nine
加一个字,我解释为什么 加上一个字一秒钟就这样

15:24 characters in the sequence in the training set now what I'd like to point out is that when you
训练组的字符序列中的字符 现在我想指出的是 当你

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

15:30 sample a chunk of data like this so say if these nine characters out of the training set
样样的一块数据, 这样表示是否训练组中的这九个字符

sample
样本 样本
神经网络基础

chunk [tʃʌŋk]
词块 词块
神经网络基础

15:36 this actually has multiple examples packed into it and that's because all of these characters
这是因为所有这些字符

15:42 follow each other and so what this thing is going to say when we plug it into a transformer
彼此相继相继相随 所以当我们把电压器插上电压器时,这东西会说什么

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

15:48 is we're going to actually simultaneously train it to make prediction at every one of these
我们实际上要同时训练它 来预测其中每一个

prediction [pri'dikʃən]
预测 模型在收到输入样本 后的输出。
神经网络基础

15:52 positions now in the in a chunk of nine characters there's actually eight individual examples packed
现身于九个字符块中的九个字符块中, 实际上有八个单个例子被包装起来

chunk [tʃʌŋk]
词块 词块
神经网络基础

16:00 in there so there's the example that when 18 when in the context of 1847 likely comes next
在那里,所以有一个例子 当18岁时 在1847年的环境下

16:08 in a context of 18 and 4756 comes next in the context of 1847 56 57 can come next and so on so
18和4756年背景下,1847 56 57年背景下。

16:17 that's the eight individual examples let me actually spell it out with code so here's a chunk of code to
八个个人例子 让我用代码来拼写 所以这里有一大块代码

chunk [tʃʌŋk]
词块 词块
神经网络基础

16:23 illustrate x are the inputs to the transformer it will just be the first block size characters y
说明 x 是变压器的输入, 它只是第一个块大小字符 y

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

16:31 will be the next block size characters so it's offset by one and that's because y are the
将是一个被一个部分抵消的, 因为y是

16:37 targets for each position in the input and then here I'm iterating over all the block size of
输入中每个位置的目标目标,然后在这里我正在对

16:43 8 and the context is always all the characters in x up to t and including t and the target is
8 上下文总是 x 中所有字符, 直至 t, 包括 t , 目标为

target ['tɑ:git]
目标 与标签 的含义相同。
神经网络基础

16:51 always the teeth character but in the targets array y so let me just run this and basically it
牙齿的性格总是有牙牙的特征 但在目标阵列中,所以让我来运行这个,基本上它

16:58 spells out what I said in words these are the eight examples hidden in a chunk of nine
上面写了我说的,这些是九个字块里 隐藏的八个例子

chunk [tʃʌŋk]
词块 词块
神经网络基础

17:04 characters that we sampled from the training set I want to mention one more thing we train on
来测试我们训练组的人物 我想再提一件事

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

17:12 all the eight examples here with context between one all the way up to context of block size and we
这里所有八个例子 都以一个到块大小的

17:19 train on that not just for computational reasons because we happen to have the sequence already
不仅因为计算的原因,我们刚好已经有了序列

17:22 or something like that it's not just done for efficiency it's also done to make the transformer
或类似的东西,它不只是做效率 它也做了制造变压器

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

17:28 network be used to seeing contexts all the way from as little as one all the way to block size
网络用于从一个小到块大小的全程观察环境

17:35 and we'd like the transformer to be used to seeing everything in between and that's going to be
我们希望变压器能习惯于 将所有东西都放在

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

17:40 useful later during inference because while we're sampling we can start to set a sampling generation
稍后在推断期间有用 因为当我们取样时 我们可以开始建立抽样组

17:45 with as little as one character of context and the transformer knows how to predict next character
上下文的字符小于一个,变压器知道如何预测下一个字符

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

17:50 with all the way up to just context of one and so then it can predict everything up to block size
直至一个环境的直角,然后它就可以预测到每个小块大小

17:55 and after block size we have to start truncating because the transformer will never receive more
块大小之后,我们不得不开始缩短时间,因为变压器将永远接收不到更多

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

18:00 than block size inputs when it's predicting the next character okay so we've looked at the
而不是块大小输入 当它预测下一个字符时, 它的区块大小输入

18:06 time dimension of the tensors that are going to be feeding into the transformer there's one
向变压器输入的 数以10 的时积

dimension
维度 维度
数学与微积分

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

18:09 more dimension to care about and that is the batch dimension and so as we're sampling these chunks of

更多的维度需要关注 那就是批量维度 因此我们抽样调查这些块

dimension
维度 维度
数学与微积分

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

18:14 text we're going to be actually every time we're going to feed them into a transformer we're going

文字我们实际上会成为 每次我们把它们注入变压器

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

18:19 to have many batches of multiple chunks of text that are all like stacked up in a single tensor

以拥有多批多个文本块,这些都像堆叠在一个发光的气体中

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

18:24 and that's just done for efficiency just so that we can keep the GPUs busy because they

这样做只是为了提高效率 这样我们就可以让GPUs保持忙碌

18:28 are very good at parallel processing of data and so we just want to process multiple chunks

数据处理的平行操作非常擅长 所以我们只想处理多个块

18:35 all at the same time but those chunks are processed completely independently they don't

全部同时同时 但是这些块子是完全独立处理的 他们没有

18:39 talk to each other and so on so let me basically just generalize this and introduce a batch dimension

彼此交谈,等等,所以让我基本上 概括一下这个,然后介绍一个批量尺寸

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

dimension
维度 维度
数学与微积分

18:44 here's a chunk of code let me just run it and then i'm going to explain what it does
这儿有一大块代码 让我来运行一下 然后解释它的作用

chunk [tʃʌŋk]
词块 词块
神经网络基础

18:51 so here because we're going to start sampling random locations in the data sets to pull chunks from
因为我们将开始抽样抽样 随机位置 在数据集中

18:56 i am setting the seed so that um in the random number generator so that the numbers i see here
我正在设置种子, 这样在随机数生成器中 um , 这样数字就在这里看到

19:02 are going to be the same numbers you see later if you try to reproduce this now the batch size
如果您尝试复制此批量大小, 将会是您以后看到的相同数字

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

19:07 here is how many independent sequences we are processing every forward backward pass of the
有多少独立的序列 我们处理的每一个前向后传

backward pass
反向传播 反向传播
神经网络基础

19:11 transformer the block size as i explained is the maximum context length to make those predictions
变压器 块大小, 如我解释的 是做出这些预测的最大上下文长度

transformer [træns'fɔ:mə]
Transformer 中文为“变换器”，但不够达雅，日常还是称英文 Transformer 中文为“变换器”，但不够达雅，日常还是称英文
神经网络基础

19:18 so let's say size four block size eight and then here's how we get batch for any arbitrary
这么说吧,四号的八号区块 然后再这样分批处理任意性

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

19:23 split if the split is a training split then we're going to look at train data otherwise at val data
如果分数是训练分成的, 分出一个部分, 那么我们将会在 Val 数据中查看列车的数据

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

19:29 that gets us the data array and then when i generate random positions to grab a chunk out
从而获得数据阵列,然后当我生成随机位置以抓取块

chunk [tʃʌŋk]
词块 词块
神经网络基础

19:36 of i actually grab i actually generate batch size number of random offsets so because this is four
随机抵消的批量大小数 所以因为这是四个

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

19:44 we are ix is going to be a four numbers that are randomly generated between zero and length of data
我们的九号将是四个数字, 随机生成在零和数据长度之间

19:50 minus block size so it's just random offsets into the training set and then x's as i explained
减区块大小,所以它只是随机的抵消 进入训练组,然后x's as I 解释

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

19:57 are the first block size characters starting at i the y's are the offset by one of that so just add
是从 i 开始的第一个块大小字符,Y是被其中的一个部分抵消的。

20:06 plus one and then we're going to get those chunks for every one of integers i in ix and use a torch
加上一个 然后我们再拿这些块子 来对每个I在九里整数

20:14 dot stack to take all those uh one dimensional tensors as we saw here and we're going to um
将所有那些... 单维的抗拉 放在我们这里看到的

20:21 stack them up at rows and so they all become a row in a four by eight tensor so here's where i'm printing

把它们堆在行堆起来,所以它们都成一排 在四乘八加八的四乘八

tensor ['tensə]

张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。

数学与微积分

20:30 them when i sample a batch xb and yb the inputs the transformer now are the input x is the four

当它们是我对批量 xb 和 Yb 进行取样时,当输入变压器现在为输入 x 是四个

sample

样本 样本

神经网络基础

batch [bætʃ]

批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。

训练与优化

20:39 by eight tensor four rows of eight columns and each one of these is a chunk of the training set

以八万条四行八列为八列,每行各为训练组块

tensor ['tensə]

张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。

数学与微积分

chunk [tʃʌŋk]

词块 词块

神经网络基础

20:48 and then the targets here are in the associated array y and they will come in to the transformer

然后目标就在这里 是在相关的数组y 中,它们会进入变压器

transformer [træns'fɔ:mə]

Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文

神经网络基础

20:54 all the way at the end to create the loss function so they will give us the correct answer

最终创造损失功能,这样他们就会给我们正确的答案

loss [lɒs]

损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M

训练与优化

21:00 for every single position inside x and then these are the four independent rows
以 X 内每个位置的单行数为单位,然后是四个独立的行

21:08 so spelled out as we did before this four by eight array contains a total of 32 examples
与我们在这四个阵列之前所做的一样,由八个阵列组成的共32个例子

21:16 and they're completely independent as far as the transformer is concerned
并且就变压器而言,他们完全独立

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

21:22 so when the input is 24 the target is 43 or rather 43 here in the y array when the input is 2443 the
当输入为 24 时, 目标值是 43 或 y 数组中43 或 43 。 当输入为 2 443 时

target ['tɑ:git]
目标 与标签 的含义相同。
神经网络基础

21:30 target is 58 when the input is 2443 58 the target is 5 etc or like when it is a 52 58 1 the
当输入为244358时,目标为5等或类似当输入为52581时

target ['tɑ:git]
目标 与标签 的含义相同。
神经网络基础

21:39 target is 58 right so you can sort of see this spelled out these are the 32 independent examples
目标58右对,所以你可以看到这个说明 这些是32个独立的例子

target ['tɑ:git]
目标 与标签 的含义相同。
神经网络基础

21:46 packed in to a single batch of the input x and then the desired targets are in y and so now this
装入输入 x 的一整批, 然后想要的目标在y 中, 所以现在这个

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

21:55 integer tensor of um x is going to feed into the transformer and that transformer is going to
微米 x 的 hunor 将装入变压器,而变压器将进入

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

22:03 simultaneously process all these examples and then look up the correct integers to predict in
同步处理所有这些示例,然后查找正确的整数以预测

22:08 every one of these positions in the tensor y okay so now that we have our batch of input that we'd
每一个位置都处于抗拉Y正常 所以现在我们有了一部分投入

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

22:14 like to feed into a transformer let's start basically feeding this into neural networks now we're going
喜欢进变压器 我们开始基本上把这个输入神经网络 现在我们要去

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

22:19 to start off with the simplest possible neural network which in the case of language modeling
以最简单的神经网络为起点,在语言建模方面,

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层 ），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

22:23 in my opinion is the bi-gram language model and we've covered the bi-gram language model in my
在我的理论中,我认为是二克语言模式 我们用我的

22:27 make more series in a lot of depth and so here i'm going to sort of go faster and let's just
更深入的系列制作, 因此在这里我要做一些更快的速度,让我们只是

22:32 implement the pytorch module directly that implements the bi-gram language model so i'm
直接执行弹火模块, 直接实施双语语言模式, 所以我

pytorch
PyTorch PyTorch
代码与工程

22:38 importing the pytorch um nn module uh for reproducibility and then here i'm constructing a bi-gram
导入 pytoch um nn 模块 uh 用于复制,然后在这里我正在建造一个双克

pytorch
PyTorch PyTorch
代码与工程

22:45 language model which is a subclass of nn module and then i'm calling it and i'm passing in the inputs
语言模式,这是 nn 模块的子类, 然后我打电话给它,然后我通过输入

对数似然 · 交叉熵 22:52

22:52 and the targets and i'm just printing now when the inputs and targets come here you see that i'm
和目标,我正在打印 现在当输入和目标 来到这里,你看到我

22:57 just taking the index uh the inputs x here which i renamed to idx and i'm just passing them
输入 x 我改名为 idx , 我只是通过它们

23:03 into this token embedding table so what's going on here is that here in the constructor we are creating
把这个象征性嵌入表放在这个符号嵌入表格中, 所以这里发生的一切就是 我们正在创建的构造器里

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

23:10 a token embedding table and it is of size vocab size by vocab size and we're using an end dot
符号嵌入表, 其大小为 vocab 的 vocaba 大小, 我们使用一个端点

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

23:17 embedding which is a very thin wrapper around basically a tensor of shape vocab size by vocab
嵌入是一个非常薄的包装, 基本上以 vocab 的形式围绕一个 数以 秒形体的 vocoab 大小

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

23:23 size and what's happening here is that when we pass idx here every single integer in our input
大小和这里发生的情况是 当我们通过 IDx 这里输入的每个整数

23:29 is going to refer to this embedding table and is going to pluck out a row of that embedding table
将引用此嵌入表格, 并要从嵌入表格中拔出一列

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

23:34 corresponding to its index so 24 here we'll go to the embedding table and we'll pluck out the 24th
和它的指数对应 所以这里24个 我们到嵌入表

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

23:40 row and then 43 we'll go here and pluck out the 43rd row etc and then pytorch is going to
和43行,然后我们到这里 拔出第43行等 然后烧火机会

pytorch
PyTorch PyTorch
代码与工程

23:47 arrange all of this into a batch by time by channel uh tensor in this case batch is 4 time is 8
全部都安排成一个批次 时间通过hhh shor频道 进行分批处理

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

23:55 and c which is the channels is vocab size or 65 and so we're just going to pluck out all those rows
频道大小是Vocab或65,所以我们只是要拔掉所有的排

24:03 arrange them in a b by t by c and now we're going to interpret this as the logits which are basically
把它们放在一个b b t t c 上 然后我们解释这个

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

24:09 the scores for the next character in a sequence and so what's happening here is we are predicting
下一个字符的分数,按顺序排列 所以这里发生的情况是 我们正在预测

24:15 what comes next based on just the individual identity of a single token and you can do that
接下来的是什么? 仅仅基于一个象征的个体身份,你可以做到

24:20 because um i mean currently the tokens are not talking to each other and they're not seeing
因为目前这些标志品 彼此之间没有交谈,也没有看到

24:25 any context except for they're just seeing themselves so i'm a i'm a token number five and then i can
除了他们只是看到自己 所以我是五号标志,然后我就可以

24:32 actually make pretty decent predictions about what comes next just by knowing that i'm token five
知道我是五号,对接下来的

24:36 because some characters uh know um sort follow other characters in in typical scenarios so we
因为有些角色 在某些典型的情景中 和其他角色相依

24:42 saw a lot of this in a lot more depth in the make more series and here if i just run this
更深入的制作系列中看到很多这些 如果我运行这个

24:48 then we currently get the predictions the scores the logits for every one of the four by eight
然后我们目前得到预测 4个中每个加8个的日志分数

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

24:54 positions now that we've made predictions about what comes next we'd like to evaluate the loss
情况已经预测了 接下来会发生什么 我们想评估损失

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

24:58 function and so in make more series we saw that a good way to measure a loss or like a quality
函数,所以在制作更多序列序列中, 我们发现这是衡量损失或质量的好方法

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

25:04 of the predictions is to use the negative log likelihood loss which is also implemented
预测的预测使用负日志概率损失,也执行这一数值

log likelihood
对数似然 The logarithm of the likelihood, used because it turns products of probabilities into sums and is often more convenient to maximize.
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

25:08 in pytorch under the name cross entropy so what we'd like to do here is loss is the
以在名称十字星号下方的烧火机 来做我们想做的就是损失

pytorch
PyTorch PyTorch
代码与工程

cross entropy
交叉熵 交叉熵
数学与微积分

25:15 cross entropy on the predictions and the targets and so this measures the quality of the logits with
预测和目标上的交叉方对子和目标的 反方在

cross entropy
交叉熵 交叉熵
数学与微积分

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

25:21 respect to the targets in other words we have the identity of the next character so how well are
换句话说,我们有下一个特性的特征,因此,如何很好

25:26 we predicting the next character based on the logits and intuitively the correct um the correct
我们根据日志预测下一个字符,直觉地预测正确的字母

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

25:33 dimension of logits uh depending on whatever the target is should have a very high number
取决于目标是什么 应该有一个非常高的数字

dimension
维度 维度
数学与微积分

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

25:38 and all the other dimensions should be very low number right now the issue is that this won't
所有其他维度应该都非常低 现在的问题是,这不会

25:44 actually this is what we want we want to basically output the logits and the loss this is what we want
这正是我们想要的, 我们想要基本上输出登入和损失 这是我们要的

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

25:51 but unfortunately uh this won't actually run we get an error message but intuitively we want to
但不幸的是,呃,这不会运行 实际上我们得到错误信息 但我们直觉上,我们希望

25:59 measure this now when we go to the pytorch cross entropy a documentation here um we're
当我们去火炬十字 向着一个文件

pytorch
PyTorch PyTorch
代码与工程

cross entropy
交叉熵 交叉熵
数学与微积分

26:07 trying to call the cross entropy in its functional form so that means we don't have to create like
试图用它的功能形式给交叉环环的 星体打电话 这样我们就不会像

cross entropy
交叉熵 交叉熵
数学与微积分

26:12 a module for it but here when we go to the documentation you have to look into the details
模块,但在这里当我们去文档时 您必须查看细节

26:17 of how pytorch expects these inputs and basically the issue here is pytorch expects if you have
皮火炬如何期望这些投入,基本上这里的问题是,如果你有的话,皮火炬预计会有

pytorch
PyTorch PyTorch
代码与工程

26:24 multi-dimensional input which we do because we have a b by t by c tensor then it actually
我们这样做是因为我们有一个b by t 乘以 c 10,然后它实际上

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

26:29 really wants the channels to be the second dimension here so if you um so basically
真正希望频道成为这里的第二个维度 所以如果你是那么...

dimension
维度 维度
数学与微积分

26:37 it wants a b by c by t instead of a b by t by c and so it's just the details of how pytorch treats
而不是b b t c 所以它只是关于火炬如何治疗的细节

pytorch
PyTorch PyTorch
代码与工程

26:46 these kinds of inputs and so we don't actually want to deal with that so what we're going to do
这样的投入,所以我们不想处理这些 所以我们要做什么

26:51 is that is we need to basically reshape our logits so here's what I like to do I like to take
我们需要基本上重塑我们的记录 所以这就是我喜欢做的

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

26:56 basically give names to the dimensions so logits that shape is b by t by c and unpack those numbers
基本上给维度提供名称, 使形状的登录为 b by t by c 并拆解这些数字

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

27:02 and then let's say that logits equals logits .vu and we want it to be a b times c b times t by c
假设登录等于记录。 似是, 我们希望它比C的 平方公尺(c) 高出1 b乘以c

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

27:10 so just a two-dimensional array right so we're going to take all the we're going to take all of these
所以,只要一个二维的阵列 右侧,所以我们要去采取 所有我们要采取 所有这些

27:18 positions here and we're going to stretch them out in a one-dimensional sequence
位置在这里,我们用一个维的顺序 把它们拉伸出来

27:23 and preserve the channel dimension as the second dimension so we're just kind of like stretching
保持频道维度作为第二维 所以我们只是有点像伸展

dimension
维度 维度
数学与微积分

27:28 out the array so it's two-dimensional and in that case it's going to better conform to what pytorch
将更符合什么是火炉

pytorch
PyTorch PyTorch
代码与工程

27:33 sort of expects in its dimensions now we have to do the same to targets because currently targets
期望的层面现在我们必须对目标也这样做,因为目前的目标

27:40 are um of shape b by t and we want it to be just b times t so one-dimensional now alternatively
微米的形状b by t 我们希望它只是 b乘以T 所以一维现在

27:49 you could always still just do minus one because pytorch will guess what this should be if you
你总是可以做一个减一个 因为Pytorch会猜到 如果你

pytorch
PyTorch PyTorch
代码与工程

27:54 want to lay it out but let me just be explicit and say pytemst. Once we've reshaped this it will
说清楚点,我说清楚一点,等我们改过它

28:00 match the cross entropy case and then we should be able to evaluate our loss okay so that right now
与交叉星盘的反环对齐 然后我们就应该能够评估损失

cross entropy
交叉熵 交叉熵
数学与微积分

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

28:08 and we can do loss and so currently we see that the loss is 4.87 now because our we have 65 possible
并且我们可以做损失,所以现在我们看到,现在的损失是4.87,因为我们有65个可能

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

28:17 vocabulary elements we can actually guess at what the loss should be and in particular
特别是,我们实际上可以猜到损失应该是什么,尤其是

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

28:23 we covered negative log likelihood in a lot of detail we are expecting log or ln of
我们用许多细节 覆盖了负日志的可能性 我们期待的日志或

log likelihood
对数似然 The logarithm of the likelihood, used because it turns products of probabilities into sums and is often more convenient to maximize.
神经网络基础

28:30 1 over 65 and negative of that so we're expecting the loss to be about 4.17 but we're getting 4.87
超过65个,负数为负 所以我们预计损失大约是4.17 但我们得到4.87

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

28:39 and so that's telling us that the initial predictions are not super diffuse they've got
也就是说,我们被告知最初的预测 不是超分散的

28:43 a little bit of entropy and so we're guessing wrong so yes but actually we're able we are able
一点点的微小 微小的 向导 所以我们猜错了 所以是的 但我们实际上能够

28:50 to evaluate the loss okay so now that we can evaluate the quality of the model on some data
评估损失是否合理,以便我们能够评价某些数据模型的质量

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

预测 · 样本 28:56

28:56 we'd like to also be able to generate from the model so let's do the generation now i'm going
我们也希望也能从模型中产生出 来,让我们做一代人 现在我要去

29:01 to go again a little bit faster here because i covered all this already in the previous videos
来再走快点 因为我在前几段视频里已经覆盖了所有这一切

29:06 so here's a generate function for the model so we take some we take the the same kind of input
所以这里为模型提供一个生成函数, 所以我们用一些, 我们使用同样的输入

29:15 idx here and basically this is the current context of some characters in a batch in some batch
IDx 这里, 基本上这就是当前某批中某些字符的上下文, 在某批中有些字符的上下下文

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

29:25 so it's also b by t and the job of generate is to basically take this b by t and extend it to be
生产的任务就是基本上把这个b t 并扩展至

29:31 b by t plus 1 plus 2 plus 3 and so it's just basically it continues the generation in all the
基本上它就是继续整个世代

29:36 batch dimensions in the time dimension so that's its job and it will do that for max new tokens
时间维度中的批量尺寸,这是它的任务, 它将做这个任务, 对于最大新标记

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

dimension
维度 维度
数学与微积分

29:42 so you can see here on the bottom there's going to be some stuff here but on the bottom
可以看到下面的底部 这里除了底部还有东西

29:46 whatever is predicted is concatenated on top of the previous idx along the first dimension which
与第一个维度相连接的

dimension
维度 维度
数学与微积分

29:53 is the time dimension to create a b by t plus 1 so that becomes a new idx so the job of generate
创建 b 乘 t+ 1 以成为新的 idx 从而生成任务的时间维度

dimension
维度 维度
数学与微积分

29:59 is to take a b by t and make it a b by t plus 1 plus 2 plus 3 as many as we want max new tokens
需要最多新牌子的, 最多要一b t 和一b t+1 + 2 + 3

30:05 so this is the generation from the model now inside the generation what we're what are we doing
所以这是我们这个代的一代 由模型组成的一代 现在在一代人中 我们在做什么

30:10 we're taking the current indices we're getting the predictions so we get those are in the logits
我们用的是当前指数 预测结果显示的预测结果 所以在日志上

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

30:17 and then the loss here is going to be ignored because we're not we're not using that and we have no
损失会被忽视 因为我们不是在使用,我们没有

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

30:23 targets that are sort of ground truth targets that we're going to be comparing with
目标,是某种地面真实目标 我们将要比较

30:28 then once we get the logits we are only focusing on the last step so instead of a b by t by c
一旦我们拿到了登账记录 我们只关注最后一步 所以不要用B比t来代替C

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

30:34 we're going to pluck out the negative one the last element in the time dimension
我们要拔出负的 时间维度中最后的元素

dimension
维度 维度
数学与微积分

30:40 because those are the predictions for what comes next so that gives us the logits which we then
因为这些是对于未来事物的预测,以便我们记录我们所记录的(消息),然后我们记录下来。

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

30:44 cover to probabilities via a softmax and then we use torsion multinomial to sample from those
以柔性负负乘的概率覆盖概率,然后我们用倍重多义质从这些样本中取样

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

sample
样本 样本
神经网络基础

30:49 probabilities and we ask pytorch to give us one sample and so idx next will become a b by 1
概率,我们要求火炬给我们一个样本 下一个就是b乘1

pytorch
PyTorch PyTorch
代码与工程

sample
样本 样本
神经网络基础

30:57 because in each one of the batch dimensions we're going to have a single prediction for what
因为每批量的每个尺寸 我们都会有一个单一的预测

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

prediction [pri'dikʃən]
预测 模型在收到输入样本 后的输出。
神经网络基础

31:01 comes next so this num samples equals 1 will make this be a 1 and then we're going to take
接下来是下一个 所以这个Num样本等于1 就会变成1 然后我们开始

31:08 those integers that come from the sampling process according to the probability distribution given here
抽样过程根据此处给出的概率分布得出的整数

distribution
分布 分布
数学与微积分

31:13 and those integers got just concatenated on top of the current sort of like running stream of integers
这些整数刚刚被集中到 目前的整数流的顶部

31:19 and this gives us a b by t plus 1 and then we can return that now one thing here is you see how
给我们一个b by t+1 然后我们可以还回去 现在有一件事

31:25 I'm calling self of idx which will end up going to the forward function I'm not providing any
我称之为自己IDx, 最终会转到我无法提供的前方功能

31:31 targets so currently this would give an error because targets is is sort of like not given
目标, 所以当前的目标, 这样目前此目标将会出错, 因为目标与未给

31:38 so targets has to be optional so targets is none by default and then if targets is none
因此,目标必须是可选择的,因此目标不是默认的,然后如果目标不是

31:44 then there's no loss to create so it's just loss is none but else all of this happens and we can
那样就不会损失了 所以只要损失是没有的... ...除了这一切发生,我们可以

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

31:51 create a loss so this will make it so if we have the targets we provide them and get a loss if
造成损失,因此,如果有目标,我们提供它们,如果

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

31:57 we have no targets it will just get the logits so this here will generate from the model
我们没有目标,它只会得到记录 这样这个模型就可以从模型中产生

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

32:04 um and let's take that for a ride now oops so I have another code chunk here which will
并且让我们把它搭上 兜兜兜风,哦哦,所以我这里还有另一个代码块

chunk [tʃʌŋk]
词块 词块
神经网络基础

32:12 generate for the model from the model and okay this is kind of crazy so maybe let me
模型生成模型的模型 并且没关系 这是种疯狂 所以也许让我

32:17 let me break this down so these are the idx right I'm creating a batch will be just one time will
让我来解开这个,这样这些就是 我制作的批量的IDx对了 只会有一次

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

32:28 be just one so I'm creating a little one by one tensor and it's holding a zero and the d type
只有一个,所以我正在创造一个小 一个逐个发压 它持有零和D类型

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

32:34 the data type is integer so zero is going to be how we kick off the generation and remember that zero
数据类型是整数, 所以零将是我们如何启动生成并记住零

32:41 is uh is the element standing for a new line character so it's kind of like a reasonable
也就是代表一个新的线性人物的元素 所以有点像一个合理的

32:46 thing to to feed in as the very first character in a sequence to be the new line
以作为第一个字符在新行的顺序中输入

32:52 so it's going to be idx which we're going to feed in here then we're going to ask for 100 tokens
之后我们再要100个象征性的

32:58 and then end that generate will continue that now because uh generate works on the level of batches
然后再结束产生后,现在将继续继续, 因为uh产生批量水平的作品

33:06 we then have to index into the zero throw to basically unplug the single batch dimension that
然后我们不得不将零投入零 键入到零投入中 基本上去插出单批尺寸

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

dimension
维度 维度
数学与微积分

33:13 exists and then that gives us a time steps just a one dimensional array of all the indices
然后我们就可以存在一个时间步骤, 只有一个维数的指数

33:22 which we will convert to simple python list from py torch tensor so that that can feed into our
转换成简单的python列表 从py火炬沙龙

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

33:28 decode function and uh convert those integers into text so let me bring this back and we're
解码功能,呃,将这些整数转换成文字 所以让我把这个带回来

33:35 generating 100 tokens let's run and uh here's the generation that we achieved so obviously
产生100个象征性的,让我们运行,呃,这里是一代 我们所实现的,显然

33:42 is garbage and the reason it's garbage is because this is a totally random model so next
垃圾是垃圾,而垃圾的原因是 因为这是一个完全随机的模型 所以下一个

33:46 up we're going to want to train this model now one more thing i wanted to point out here is
上方我们想训练这个模型 现在还有一件事我想指出

33:52 this function is written to be general but it's kind of like ridiculous right now because
此函数是写作为一般的, 但现在有点像荒谬,因为

33:57 we're feeding in all this we're building out this context and we're concatenating it all
我们用所有这些食物来喂养我们, 我们正在构建这个背景 而我们正在将这一切

34:02 and we're always feeding it all into the model but that's kind of ridiculous because this is
我们总是把一切都注入模型 但是这有点荒谬 因为

34:08 just a simple bygram model so to make for example this prediction about k we only needed this w
简单一个字法模型, 以便举例来预测 k, 我们只需要这个 w

prediction [pri'dikʃən]
预测 模型在收到输入样本 后的输出。
神经网络基础

34:14 but actually what we fed into the model is we fed the entire sequence and then we only looked at the
但实际上我们输入到模型中 是我们输入了整个序列 然后我们只查看

34:18 very last piece and predicted k so the only reason i'm writing it in this way is because
最后一块,预测K 所以,我写这个的唯一原因 是因为

34:25 right now this is a bygram model but i'd like to keep this function fixed and i'd like it to work
现在这个是按语法模型 但我想保持这个功能固定 我希望它能工作

34:31 later when our characters actually basically look further in the history and so right now the
等我们的角色后来实际上 基本上更进一步看历史

34:37 history is not used so this looks silly but eventually the history will be used and so that's
历史没有被使用,所以这看起来很傻 但最终历史会被使用 所以

34:43 why we want to do it this way so just a quick comment on that so now we see that this is random so let's

为什么我们想这样做,所以只要简单简要地评论一下,现在我们发现这是随机的,所以让我们

34:50 train the model so it becomes a bit less random okay let's now train the model so first what i'm

训练模型,这样它就会变得少一点随机的

图形处理单元 · 训练循环 34:55

34:55 going to do is i'm going to create a pytorch optimization object so here we are using the

将要做的是,我要建立一个火炬优化对象 所以我们在这里使用

pytorch
PyTorch PyTorch
代码与工程

35:00 optimizer adam w now in a make more series we've only ever used the cast a gradient descent

最佳阿坦现在在一系列的系列中 我们只用过梯度下坡

optimizer
优化器 梯度下降法 的一种具体实现。 TensorFlow 的优化器基类是 tf.train.Optimizer (https://www.tensorflow.org/api_docs/python/tf/train/Optimizer)。不同的优化器 可能会利用以下一个或多个概念来增强梯度下降法在指定 训练集 中的效果：
训练与优化

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

35:07 the simplest possible optimizer which you can get using the sgd instead but i want to use

最简单的优化器,您可以使用 sgd 代替 Sgd,但我想使用

optimizer
优化器 梯度下降法 的一种具体实现。 TensorFlow 的优化器基类是 tf.train.Optimizer (https://www.tensorflow.org/api_docs/python/tf/train/Optimizer)。不同的优化器 可能会利用以下一个或多个概念来增强梯度下降法在指定 训练集 中的效果：
训练与优化

35:12 adam which is a much more advanced and popular optimizer and it works extremely well for a typical
亚坦是一个更先进和流行的优化剂,对于典型的

optimizer
优化器 梯度下降法 的一种具体实现。 TensorFlow 的优化器基类是 tf.train.Optimizer (https://www.tensorflow.org/api_docs/python/tf/train/Optimizer)。不同的优化器 可能会利用以下一个或多个概念来增强梯度下降法在指定 训练集 中的效果：
训练与优化

35:18 good setting for the learning rate is roughly three negative four but for very very small networks like
学习率的良好环境大约是三张负四张,但对于非常小的网络来说,例如:

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

35:23 it's the case here you can get away with much much higher learning rates one in negative three or
情况是这样的 你可以在高得多的高学习率中 以负数三分之一或

35:27 even higher probably but let me create the optimizer object which will basically take the
更可能更高 但让我来创造一个优化对象 这个物体基本上会

optimizer
优化器 梯度下降法 的一种具体实现。 TensorFlow 的优化器基类是 tf.train.Optimizer (https://www.tensorflow.org/api_docs/python/tf/train/Optimizer)。不同的优化器 可能会利用以下一个或多个概念来增强梯度下降法在指定 训练集 中的效果：
训练与优化

35:33 gradients and update the parameters using the gradients and then here our batch size up above
梯度,然后用渐渐度更新参数,然后在这里显示我们上方的批量大小

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

35:40 was only four so let me actually use something bigger let's say 32 and then for some number of steps
仅四点,所以让我用更大的 比32个再加几步

35:46 we are sampling a new batch of data we're evaluating the loss we're zeroing out all the
我们正在抽样 新的一批数据 我们正在评估的损失

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

35:52 gradients from the previous step getting the gradients for all the parameters and then using
从前一步获得所有参数的梯度并随后使用

35:56 those gradients to update our parameters so typical training loop as we saw in the make
这些梯度可以更新参数 从而像我们在制造中看到的典型训练环

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

36:01 more series so let me now run this for say 100 iterations and let's see what kind of losses
更多系列,所以现在让我来做100次重复, 让我们看看什么样的损失

36:07 we're gonna get so we started around 4.7 and now we're going to down to like 4.6 4.5 etc so
我们得到的,我们开始约 4.7,现在我们 下降到像4.64.5等等

36:17 the optimization is definitely happening but let's sort of try to increase number of iterations
优化绝对正在发生 但我们试着增加迭代次数

36:24 and only print at the end because we probably will not train for longer okay so we're down to 3.6
最终只打印 因为可能我们不会训练更长的时间

36:31 roughly roughly down to three this is the most janky optimization okay it's working let's just
大约大约到3个 这是最强基优化的

36:48 do 10 000 and then from here we want to copy this and hopefully we're going to get something reasonable
10000拍10000 然后从这里 我们想复制这个 希望我们能找到一些合理的东西

36:57 and of course it's not going to be Shakespeare from by Graham model but at least we see that the
当然,它不会是莎士比亚 从格雷厄姆模型的莎士比萨 但至少我们看到

37:01 loss is improving and hopefully we're expecting something a bit more reasonable
损失在改善 希望我们期待一些更合理的

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

37:07 okay so we're down at about 2.5 ish let's see what we get okay dramatic improvement certainly
OK,所以我们下降在2.5 yah左右 让我们看看什么 我们得到没事的 显著改善肯定

37:13 on what we had here so let me just increase the number of tokens okay so we see that we're starting
来,让我们在这里的 我们在这里拥有的东西 所以让我只增加数量 象征性的数

37:20 to get something at least like reasonable ish um certainly not Shakespeare but the model is making
至少要得到一些 合理的东西,嗯 嗯当然不是莎士比亚 但模型正在制作

37:29 progress so that is the simplest possible model so now what I'd like to do is obviously that
所以这是最简单的模式 所以我现在想做的是

37:38 this is a very simple model because the tokens are not talking to each other so given the previous
这是一个非常简单的模型, 因为之前的标志品没有相互交谈

37:42 context of whatever was generated we're only looking at the very last character to make the
任何生成的外在背景 我们只看最后一个字符

37:47 predictions about what comes next so now these now these tokens have to start talking to each other
预测接下来会发生什么 所以现在这些象征物 开始互相交谈

37:52 and figuring out what is in the context so that they can make better predictions for what
并找出背景中的内容,以便他们能够对什么作出更好的预测

37:56 comes next and this is how we're going to kick off the transformer okay so next I took the
接下来我们接下来就来,这样我们就可以 启动变压器了,所以我接着拿了

transformer [træns'fɔ:mə]
Transformer 中文为“变换器”，但不够达雅，日常还是称英文 Transformer 中文为“变换器”，但不够达雅，日常还是称英文
神经网络基础

38:01 code that we developed in this jupy notebook and I converted it to be a script and I'm doing
我们在这个 Jupy 笔记本里开发的代码 我把它转换成一个脚本,我正在做

38:06 this because I just want to simplify our intermediate work which is just the final
因为我只想简化我们中间工作,这是最后的

38:10 product that we have at this point so in the top here I put all the hyperparameters that we
产品,我们在这个时候有的 产品,所以在顶部 在这里我把所有的超参数

38:16 defined I introduced a few and I'm going to speak to that in a little bit otherwise a lot of this
定义我介绍了一些,我要说到这一点 在一点点,否则很多这样的

38:20 should be recognizable reproducibility read data get the encoder and the decoder create the
数据获取编码器,解码器创建

38:27 train and test splits I use the kind of like data loader that gets a batch of the inputs
和测试列车分割我使用一种类似数据装载器的输入量

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

38:34 and targets this is new and I'll talk about it in a second now this is the bi-gram language
目标 这是新的,我马上再谈 现在这是二字语言

38:40 model that we developed and it can forward and give us a logits and loss and it can generate
能够向前推进,给我们一个日志和损失,并产生

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

38:46 and then here we are creating the optimizer and this is the training loop so everything here should
然后在这里,我们创造优化的优化器 这就是训练循环 所以这里的一切应该

optimizer
优化器 梯度下降法 的一种具体实现。 TensorFlow 的优化器基类是 tf.train.Optimizer (https://www.tensorflow.org/api_docs/python/tf/train/Optimizer)。不同的优化器 可能会利用以下一个或多个概念来增强梯度下降法在指定 训练集 中的效果：
训练与优化

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

38:52 look pretty familiar now some of the small things that I added number one I added the ability to
现在看起来很熟悉了 有些小东西 我加了第一件

38:58 run on a GPU if you have it so if you have a GPU then you can this will use CUDA instead of just
如果您有 GPU , 您可以使用 CUDA 而不是

gpu
图形处理单元/图形处理器 图形处理单元/图形处理器
神经网络基础

39:04 CPU and everything will be a lot more faster now when device becomes CUDA then we need to make
CPU和所有东西都更快 现在当装置变成CUDA时

cpu
中央处理器 中央处理器
神经网络基础

device [di'vais]
设备 一类可运行 TensorFlow 会话的硬件，包括 CPU 、 GPU 和 TPU 。
神经网络基础

39:10 sure that when we load the data we move it to device when we create the model we want to move
确定当我们装入数据时, 当我们创建模型时, 我们移动到设备中, 我们想要移动

device [di'vais]
设备 一类可运行 TensorFlow 会话的硬件，包括 CPU 、 GPU 和 TPU 。
神经网络基础

39:16 the model parameters to device so as an example here we have the in an embedding table and it's
设备模型参数, 以此为例, 我们在一个嵌入表格中, 它被

device [di'vais]
设备 一类可运行 TensorFlow 会话的硬件，包括 CPU 、 GPU 和 TPU 。
神经网络基础

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

39:22 got a dot weight inside it which stores the sort of lookup table so that would be moved to the
里面有一个点重量, 它存储了一种查看表格, 这样就可以移动到

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

39:28 GPU so that all the calculations here happen on the GPU and they can be a lot faster and then
GPU 这样所有的计算都发生在 GPU上 并且它们可以更快,然后

gpu
图形处理单元/图形处理器 图形处理单元/图形处理器
神经网络基础

39:34 finally here when I'm creating the context that feeds into generate I have to make sure
最后,当我在这里 创造环境 将它转化为产生时,我必须确定

39:38 that I create on the device number two what I introduced is the fact that here in the training
我发明在第二台装置上 我介绍的是 训练中

device [di'vais]
设备 一类可运行 TensorFlow 会话的硬件，包括 CPU 、 GPU 和 TPU 。
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

39:45 loop here I was just printing the loss dot item inside the training loop but this is a very noisy
环这里我刚刚在训练圈里打印了丢失点项目 但是这很吵

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

39:53 measurement of the current loss because every batch will be more or less lucky and so what I
衡量目前损失的计量,因为每批货物或多或少都算幸运,所以我

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

39:59 want to do usually is I have an estimate loss function and the estimate loss basically then
通常想要做的是 我有估计损失功能 估计损失基本上就是

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

40:07 goes up here and it averages up the loss over multiple batches so in particular we're going to
平均损失增加多批量 所以特别要说

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

40:15 iterate eval-iter times and we're going to basically get our loss and then we're going to get the
自动电极时代 我们基本上会损失损失 然后我们就会得到

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

40:20 average loss for both splits and so this will be a lot less noisy so here when we call the
两者平均损失的平分 所以这将会少得多吵闹 所以当我们叫

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

40:26 estimate loss we're going to report the pretty accurate train and validation loss
估计损失,我们报告 相当准确的火车和验证损失

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

validation
验证 验证
训练与优化

40:30 now when we come back up you'll notice a few things here I'm setting the model to evaluation phase
等我们回来的时候 你会注意到这里的一些事情 我正在设置评估阶段的模型

40:37 and down here I'm resetting it back to training phase now right now for our model as is this
下面我正在重新把它 恢复到训练阶段 现在,我们这个模范

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

40:42 doesn't actually do anything because the only thing inside this model is this and end up embedding
之所以没有做什么,其实是因为这个模型中的唯一东西 就是这个,最终嵌入

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

40:48 and this this network would behave both would be have the same in both evaluation mode and
并且这个网络的行为方式 在评价模式和

40:55 training mode we have no dropout layers we have no bathroom layers etc but it is a
培训模式 我们没有辍学层 我们没有浴室层等 但是它是一个

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

dropout
随机失活 随机失活
训练与优化

41:00 good practice to think through what mode your neural network is in because some layers will
思考你神经网络的形态,因为有些层次

neural network
神经网络 一种模型，灵感来源于脑部结构，由多个层构成（至少有一个是隐藏层），每 个层都包含简单相连的单元或神经元 （具有非线性关系）。
神经网络基础

41:05 have different behavior at inference time or training time and there's also this context
推论时间或训练时间的行为不同, 也有这种背景

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

41:12 manager torched up no grad and this is just telling PyTorch that everything that happens inside this
经理把没有研究生的毕业生烧了 这只是告诉皮托尔奇

pytorch
PyTorch PyTorch
代码与工程

41:16 function we will not call dot backward all and so PyTorch can be a lot more efficient with
我们不会将点后向调全部, 功能不会调用点后向, 所以PyTorrch 可以提高效率

pytorch
PyTorch PyTorch
代码与工程

41:22 its memory use because it doesn't have to store all the intermediate variables because
因为它不需要存储所有中间变量 因为

41:26 we're never going to call backward and so it can it can be a lot more efficient in that way
我们永远不会叫后退 所以它能提高效率

41:31 so also a good practice to tell PyTorch when we don't intend to do back propagation so right now
这也是一个很好的做法 当我们不想做后台时 告诉皮托尔奇

pytorch
PyTorch PyTorch
代码与工程

back propagation
反向传播 反向传播
神经网络基础

矩阵乘法 · 矩阵 41:38

41:38 the script is about 120 lines of code of and that's kind of our starter code I'm calling it
剧本大约120行的代码线 这是种我们最初的代码

41:45 by gram dot pi and I'm going to release it later now running this script gives us output in the
后我再发布它 现在运行这个脚本 给我们在

41:52 terminal and it looks something like this it basically as I ran this code it was giving me the
终端看起来像这样的,基本上我运行这个代码的时候 它给了我一个

41:58 train loss and the val loss and we see that we convert to somewhere around 2.5 with the by-grad model

火车和val损失,我们看到我们转换成2.5左右的 路程模型

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

42:04 and then here's the sample that we produced at the end and so we have everything packaged up in

然后这里就是我们最后生产的样本, 我们把所有的东西包装起来

sample
样本 样本
神经网络基础

42:10 the script and we're in a good position now to iterate on this okay so we are almost ready

剧本和我们处于一个好位置 现在可以循环了,好吗? 好,所以我们几乎准备好了

42:14 to start writing our very first self- attention block for processing these tokens now before we

开始写下我们第一个自我注意的屏障, 以便处理这些象征性物。

attention
注意力 注意力
神经网络基础

42:22 actually get there I want to get you used to a mathematical trick that is used in the self- attention

真正到达那里,我想让你习惯一个数学技巧 用来自我注意

attention
注意力 注意力
神经网络基础

42:27 inside a transformer and it's really just like at the heart of an an efficient implementation

变压器内部的变压器里, 它真的就像一个高效实施的核心

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

42:32 of self- attention and so I want to work with this toy example to just get you used to this

自我注意 所以我想和这个玩具范例合作 让你习惯

attention
注意力 注意力
神经网络基础

42:37 operation and then it's going to make it much more clear once we actually get to to it
操作,然后它会更清晰 等我们真正进入

operation
运算 运算
神经网络基础

42:43 in the script again so let's create a b by t by c where b, t, and c are just 4, 8, and 2 in the story
剧本中再次出现,所以让我们创建一个b by t t by c, 其中 b, t, c 仅是4, 8和 2 在故事中

42:49 example and these are basically channels and we have batches and we have the time component
举例来说,这些基本上就是管道, 我们有分批的分批, 我们有时间部分

42:56 and we have some information at each point in the sequence so c now what we would like to do
并且我们在每个时间点都有信息 在顺序上,所以现在

43:03 is we would like these tokens so we have up to eight tokens here in a batch and these eight
我们想要这些礼物 所以我们有八个 批量的

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

43:09 tokens are currently not talking to each other and we would like them to talk to each other we'd
等物牌目前不互相交谈 我们希望他们彼此交谈

43:12 like to couple them and in particular we don't we we want to couple them in a very specific way
特别我们不想以非常具体的方式 来对夫妻

43:19 so the token for example at the fifth location it should not communicate with tokens in the sixth
等符号,例如,在第五处的代号,它不应该与第六处的代号和第六处的

43:25 seventh and eighth location because those are future tokens in the sequence the token on a
第七位和第八位位置,因为这些是未来标志,在顺序中,这些标记

43:31 fifth location should only talk to the one in the fourth third second and first so it's only
第五位置应该只跟 第四三三秒的

43:37 information only flows from previous context to the current time step and we cannot get any
信息仅从先前的背景流向当前时间步骤,我们无法获得任何

43:42 information from the future because we are about to try to predict the future so
未来的信息 因为我们即将尝试预测未来

43:47 what is the easiest way for tokens to communicate okay the easiest way I would say is okay if we
最简单的方式是什么? 最简单的方式,令象征性物品交流 OK

43:54 are up to if we're a fifth token and I'd like to communicate with my past the simplest way
以第五代的象征,我想 用最简单的方式 与我的过去沟通

43:58 we can do that is to just do an average of all the of all the preceding elements
我们能够做到这一点,就是仅仅做一个以上所有要素的平均数

44:06 so for example if I'm the fifth token I would like to take the channels that make up that are
例如,如果我是第五件礼物 我想通过 构成

44:12 information at my step but then also the channels from the fourth step third step second step in
以及第四步第三步第二步的渠道

44:16 the first step I'd like to average those up and then that would become sort of like a feature
第一步我想把那些平均起来 然后它就会变成一个特征

44:21 vector that summarizes me in the context of my history now of course just doing a sum or
将我总结在历史背景中的矢量 现在当然只是做一个总和或

vector
向量 向量
数学与微积分

44:27 like an average is an extremely weak form of interaction like this communication is
平均是极弱的相互作用形式 就像这种交流

44:31 extremely lossy we've lost a ton of information about spatial arrangements of all those tokens
极其损失的非常严重 我们失去了一吨信息 关于空间安排的信息 所有这些象征性物

44:37 but that's okay for now we'll see how we can bring that information back later
不过现在没关系了,我们看看 以后如何把信息带回来

44:40 for now what we would like to do is for every single batch element independently
现在,我们想做的是 独立地对每一批的成分

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

44:46 for every teeth token in that sequence we'd like to now calculate the average of all the
每一个牙齿标记的 牙印 在那个序列中的每个牙齿符号 现在我们要计算

44:52 vectors in all the previous tokens and also at this token so let's write that out
之前所有要标的矢量, 以及此要标的向量, 所以让我们写出来

44:59 um I have a small snippet here and instead of just fumbling around let me just copy paste it and talk to
我这里有个小片子 而不是随便乱翻 让我抄一下贴上它然后和他谈谈

45:04 it so in other words we're going to create x and the b o w is short for bag of words because bag of
换换换换句话说,我们将会创造xx, 而bow是短的一袋单词,因为包

45:12 words is is kind of like a term that people use when you are just averaging up things so this
文字就像人们使用的一个术语 当你只是平均做一些事情,所以

45:19 is just a bag of words basically there's a word stored on every one of these eight locations
基本上这八个位置中的每一个位置 都储存着一个字

45:24 and we're doing a bag of words are just averaging so in the beginning we're going to say that it's
我们做的就是一袋单词 平均平均,所以在开始的时候 我们要说

45:29 just initialized at zero and then I'm doing a for loop here so we're not being efficient yet that's
零时刚初始化,然后我在这里做一个循环 所以我们效率不高

45:33 coming but for now we're just iterating over all the batch dimensions independently iterating over
至今我们只是对所有批量尺寸的循环 独立地循环

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

45:38 time and then the previous tokens are at this batch a dimension and then everything up to and
时间和时间,然后之前的标记在此批量中是一个维度 然后所有事情都到

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

dimension
维度 维度
数学与微积分

45:47 including the teeth token okay so when we slice out x in this way x-prev becomes of shape
包括牙齿标记在内,可以这样等当我们这样切掉 x 这样X -prev 变成形状

45:56 how many t elements there were in the past and then of course c so all the two-dimensional
过去有多少个元素 t , 然后当然是c 所以所有的二维

46:02 information from these little tokens so that's the previous sort of chunk of tokens from my
这些小标记中的信息信息, 所以这就是我先前的 类似标记的块 从我的

chunk [tʃʌŋk]
词块 词块
神经网络基础

46:10 current sequence and then I'm just doing the average or the mean over the zero dimension so
目前的顺序,然后我做平均或平均值 在零维上

dimension
维度 维度
数学与微积分

46:15 I'm averaging out the time here and I'm just going to get a little c one-dimensional vector which
我平均时间在这里,我 将只是要去得到一个小 的一维矢量

vector
向量 向量
数学与微积分

46:21 I'm going to store in x-bag of words so I can run this and this is not going to be very informative
我要用X包的字来储存 这样我就可以运行了 这不会很丰富

46:28 because let's see so this is x of zero so this is the zero batch element and then x-bow at zero
因为让我们来看看,这是零xx0 所以这是零批次元素,然后是零时X弓

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

46:37 now you see how the at the first location here you see that the two are equal and that's
现在你可以看到,在第一个位置 在这里你看到两个是平等的

46:44 because it's we're just doing an average of this one token but here this one is now an average of
因为我们只是做一个普通的这个象征 但现在这个是

46:50 these two and now this one is an average of these three and so on so and this last one is the average
这两张,现在这个是这三个的平均数,等等。 最后一张是平均值

47:01 of all of these elements so vertical average just averaging up all the tokens now gives this
所有这些元素中, 垂直平均均数的垂直均数 都平均了所有现在给定的符号

47:06 outcome here so this is all well and good but this is very inefficient now the trick
结果是,这都是好而好的,但现在的技巧是效率非常低

47:12 is that we can be very very efficient about doing this using matrix multiplication so that's the
也就是说,我们可以用矩阵乘法来非常高效地完成这个任务, 这样就可以

matrix
矩阵 矩阵
数学与微积分

47:18 mathematical trick and let me show you what I mean let's work with the toy example here let me run it
数学把戏 让我来告诉你我的意思 让我们用玩具的例子来做吧 让我来操作

47:23 and I'll explain I have a simple matrix here that is a three by three of all the ones a matrix B
我会解释我有一个简单的矩阵 是一个三乘三的矩阵 B

matrix
矩阵 矩阵
数学与微积分

47:30 of just random numbers and it's a three by two and a matrix C which will be three by three
3乘2 和C矩阵,3乘3

matrix
矩阵 矩阵
数学与微积分

47:35 multiplied three by two which will give out a three by two so here we're just using um
乘以三乘以二 就会出一个三乘二 所以我们在这里只是用

47:41 matrix multiplication so a multiply B gives us C so how are these numbers in C
乘以矩阵矩阵乘法,乘以B 给我们C 所以这些数字怎么用C

matrix
矩阵 矩阵
数学与微积分

47:51 achieved right so this number in the top left is the first row of a dot product with the first
右实现, 因此左上左上方的这个数字是第一个点产品第一行

dot product
点积 点积
数学与微积分

47:58 column of B and since all the the row of A right now is all just once then the dot product here
B列,因为现在A列的所有 A行 都只是一次,然后点产品在这里

dot product
点积 点积
数学与微积分

48:04 with with this column of B is just going to do a sum of these of this column so two plus six
与 B 列的本列 相加 B 列 将只是做一整数的总和 所以二加六

48:10 plus six is 14 the element here in the output of C is also the first column here the first row of A
+6 + 6 是 14 C 输出中元素的此处是 C 输出中的元素, 这里也是 A 第一列的第一列, 这里就是 A 第一行

48:18 multiplied now with the second column of B so seven plus four plus five is 16 now you see that there's
乘以B第二列,7+4+5 16,现在你可以看到

48:25 repeating elements here so this 14 again is because this row is again all once and it's multiplying
这里重复元素, 所以这14 是因为这个行再次是一次, 它正在乘

48:30 the first column of B so we get 14 and this one is and so on so this last number here is the
B第一列,所以我们得到14,这个是等等。 最后的号码是

48:36 last row dot product last column now the trick here is the following this is just a boring number of
最后一个行点产品最后一列最后一列, 现在这里的技巧是下面的这个只是一个无聊的数字

dot product
点积 点积
数学与微积分

48:46 it's just a boring array of all ones but torche has this function called trill which is short
这只是一个无聊的阵列 所有的那些,但火炬有 这个功能叫做三轮,短

48:52 for a triangular something like that and you can wrap it in torche dot ones and we'll just
三角形的三角形,你可以用火炬点的 东西包起来,我们只是

48:58 return the lower triangular portion of this okay so now it will basically zero out these guys here
返回此部分的下三角形部分, 可以吗? 现在基本上他们在这里零

49:07 so we just get the lower triangular part well what happens if we do that so now we'll have
所以,我们只是得到 较低的三角三角下方部分好 如果我们这样做会发生什么 如果这样,现在我们将有

49:16 A like this and B like this and now what are we getting here in C well what is this number
像这样的A和B,像这样的现在 我们在这里C得到什么? 这个数字是什么?

49:21 well this is the first row times the first column and because this is zeros
这是第一行乘第一列的第一行, 因为这是零

49:26 these elements here are now ignored so we just get a two and then this number here is the first row
元素被忽略了,所以我们只得到两个 然后这个数字是第一排

49:34 times the second column and because these are zeros they get ignored and it's just seven the seven
第二列乘以第二列,因为这些都是零,所以被忽略了, 7是7

49:40 multiplies this one but look what happened here because this is one and then zeros we what ended
乘以这个倍数,但看看这里发生了什么 因为这是一零 我们结束了

49:46 up happening is we're just plucking out the row of this row of B and that's what we got
发生的事情就是 我们只是从这排B中抽出一排

49:51 now here we have 110 so here 110 dot product with these two columns will now give us two plus six
这儿有110个点产品,这里有110个。 110个点的产品与这两栏的两列 将给我们2加6

dot product
点积 点积
数学与微积分

50:00 which is eight and seven plus four which is 11 and because this is 111 we ended up with the addition
也就是8和7+4, 11和11, 并且因为这是111 我们最后加了

50:07 of all of them and so basically depending on how many ones and zeros we have here we are basically
基本上取决于我们这里有多少个一和零, 我们基本上是

50:13 doing a sum currently of the variable number of these rows and that gets deposited into C
正在做一个和这些行变量数的当前和存入 C

50:21 so currently we're doing sums because these are ones but we can also do average right and you can
因为我们现在在做一些金额,因为这些都是那些, 但我们也可以做平均正确的事情,你可以

50:26 start to see how it could do average of the rows of B sort of in incremental fashion because we
开始看看它如何能以递增方式 平均完成B行的平均列数

50:34 don't have to we can basically normalize these rows so that they sum to one and then we're going to
不用我们就可以把这些行都正常化 这样他们就能一对一 然后我们就

50:38 get an average so if we took A and then we did A equals A divide a torch dot sum in the
平均得到平均,所以如果我们拿A 然后我们做了A等A 将点数和火炬点数分开

50:47 of A in the um one dimension and then let's keep them is true so therefore the broadcasting will
保持它们的真实性 所以广播

dimension
维度 维度
数学与微积分

50:57 work out so if I rerun this you see now that these rows now sum to one so this row is one this
工作,所以如果我重新运行这个... ...现在你可以看到这些行现在一对一地加起来 所以这行就是一

51:05 rose point five point five is zero and here we get one thirds and now when we do a multiply B
升起点5点5分5分零是0,这里我们得到三分之一 现在当我们做乘以B时

51:10 what are we getting here we are just getting the first row first row here now we are getting the
我们刚拿到第一排第一排的

51:16 average of the first two rows okay so two and six average is four and four and seven average is five
前两行平均数前两排平均数 行行 好 所以二六平平均数是四 四 四个 七平均数是五

51:24 point five and on the bottom here we are now getting the average of these three rows so the
下方的三排平均数 所以

51:31 average of all of elements of B are now deposited here and so you can see that by manipulating
B 中所有元素的平均均值, 现在都在此存储, 这样您可以看到, 通过操控

51:39 these elements of this multiplying matrix and then multiplying it with any given matrix we can do
these
乘数矩阵表的这些元素,然后与任何特定的矩阵表一起乘以。

matrix
矩阵 矩阵
数学与微积分

51:46 averages in this incremental fashion because we just get um and we can manipulate that based on
平均平均值,以这种渐进的方式 因为我们只是得到嗯 我们可以操纵基于

51:53 the elements of A okay so that's very convenient so let's swing back up here and see how we can
OK的元素 所以这非常方便 所以我们回这里看看我们如何

51:58 vectorize this and make it much more efficient using what we've learned so in particular
将这种传介化, 并使它更高效地使用我们学到的东西, 特别是

52:03 we are going to produce an array A but here I'm going to call it way short for weights but this is
我们将要制作一个A组,但是在这里,我把它称为重量短,但这是

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

52:09 our A and this is how much of every row we want to average up and it's going to be an average
我们的A,这是我们想要平均 每一行中有多少排 我们希望平均

52:16 because you can see that these rows sum to one so this is our A and then our B in this example
因为你可以看到这些行和一对一 所以这是我们的A,然后是B 在这个例子中

52:23 of course is X so it's going to happen here now is that we are going to have an expo two
当然是X 所以现在这里将要发生的事情 就是我们有一个2号展览

52:30 two and this expo two is going to be way multiplying rx so let's think this through way is t by t and
和这个2号展览会将是方式 乘以Rx 所以让我们想 通过的方式T t

52:42 this is matrix multiplying in pytorch a B by t by C and it's giving us uh what shape so pytorch
这是以C乘以B乘以B的 火炉乘以C的基质, 它给我们提供了什么形状?

matrix
矩阵 矩阵
数学与微积分

pytorch
PyTorch PyTorch
代码与工程

52:52 will come here and it will see that these shapes are not the same so it will create a bash
就会来到这里,它会看到这些形状不同 所以它会制造一个振动

52:56 dimension here and this is a batch matrix multiply and so it will apply this matrix multiplication
这里的尺寸, 这是批量矩阵表乘法, 将应用此矩阵表乘数

dimension
维度 维度
数学与微积分

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

53:03 in all the batch elements um in parallel and individually and then for each batch element
在所有批次元素中,同时并单地和单独地,然后对每一批次元素

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

53:09 there will be a t by t multiplying t by c exactly as we had below so this will now create B by t by
将有一个t t,通过t乘 t,c 精确的乘 t,C 我们下面有完全一样 因此它现在创造B

53:19 C and expo two will now become identical to expo so we can see that torched dot all close
C和2号展会现在将 与展览会完全相同 这样我们就能看到那被烧光的点 几乎接近

53:31 of expo and expo two should be true now so this kind of like convinces us that these are in fact
博览和2号展览和2号展出现在应该是真的了 所以这种像说服我们

53:41 the same so expo and expo two if i just print them uh okay we're not going to be able to okay
同样的展览和2博览会 如果我只是打印它们 嗯好吧 我们无法做到

53:50 we're not going to be able to just stare it down but well let me try expo basically just at the
我们无法只盯着它看下去... ...但让我尝试博览会,基本上

53:56 zero element and expo two at the zero element so just the first batch and we should see that this
零元素零,二号实验在零元素零 所以只要第一批 我们就能看

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

54:02 and that should be identical which they are right so what happened here the trick is we were able
并且应该完全相同,他们是正确的 他们是正确的 所以这里发生的一切

54:08 to use batch matrix multiply to do this uh aggregation really and it's a weighted aggregation
使用批量矩阵乘以 来完成这个 uh 汇总, 它是一个加权汇总

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

matrix
矩阵 矩阵
数学与微积分

54:16 and the weights are specified in this um t by t array and we're basically doing weighted sums
重量由 t 数组来指定, 我们基本上在做加权总和

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

54:23 and uh these weighted sums are according to the weights inside here that take on sort of this
这些加权总和 与这里的重量

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

54:30 triangular form and so that means that a token at the teeth dimension will only get
三角形,因此这意味着牙牙牙尺寸的标记只会得到

dimension
维度 维度
数学与微积分

54:36 sort of information from the tokens preceding it so that's exactly what we want and finally
之前的标记上的信息种类 这正是我们想要的 最后

54:42 i would like to rewrite it in one more way and we're going to see why that's useful so this is
我想再重写一个字,我们看看为什么有用

54:48 the third version and it's also identical to the first and second but let me talk through it it uses
第三版,和第一版和第二版完全相同 不过让我通过它来谈谈

54:54 softmax so trill here is this matrix lower triangular ones way begins as all zero okay so if
软软最大软最大软max so tritll 这里的三角矩阵是这个基质 下三角方形的矩阵 开始为零

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

matrix
矩阵 矩阵
数学与微积分

55:06 i just print way in the beginning it's all zero then i used masked fill so what this is doing is
一开始我用面具填充 所以这是

55:16 weight that masked fill it's all zeros and i'm saying for all the elements where trill is equal to
代表所有元素的三进数等于

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

55:21 equals zero make them be negative infinity so all the elements where trill is zero will become
零等于零, 使其为负无无限度, 所以三进制为零的所有元素都会变成

55:28 negative infinity now so this is what we get and then the final one here is softmax so if i
负无限,所以这就是我们得到的 最后的是软最大

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

55:37 take a softmax along every single so dim is negative one so along every single row if i do a softmax
每一个暗暗的都带软式的 软式的,每行都是负的 如果我做了软式的

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

55:44 what is that going to do well softmax is um it's also like a normalization operation right and so
这样做能做的很好软式的 um,它也就像正常行动

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

55:54 spoiler alert you get the exact same matrix let me bring back the softmax and recall that in
破坏者警告你得到的 和相同的矩阵 让我把软麦拿回来

matrix
矩阵 矩阵
数学与微积分

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

56:02 softmax we're going to exponentiate every single one of these and then we're going to divide
将每一个这些的 排在前面 然后我们分

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

56:06 by the sum and so if we exponentiate every single element here we're going to get a one and here we're
以总和来计算,所以如果我们把这里的每一个元素都加到一个

56:12 going to get uh basically zero zero zero zero everywhere else and then when we normalize we just get
基本上到零零零零 零零零,然后当我们恢复正常 我们只是得到

56:19 one here we're going to get one one and then zeros and then softmax will again divide and this
其中一个在这里,我们将会得到一个和零 然后零 然后软峰值会再次分裂

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

56:25 will give us point five point five and so on and so this is also the the same way to produce
给我们5点5点5,等等,所以这也是同样的生产方式

56:31 this mask now the reason that this is a bit more interesting and the reason we're going to end
和我们即将结束的原因

56:36 up using it in self-attention is that these weights here begin with zero and you can think of this as
自我注意时使用它,就是这些加权数 从零开始,你可以把它想成

attention
注意力 注意力
神经网络基础

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

56:44 like an interaction strength or like an affinity so basically it's telling us how much of each
像是互动的力量 或者像亲近 所以基本上它告诉我们

56:51 token from the past do we want to aggregate an average up and then this line is saying
是否想要从过去开始, 我们想要将平均的上下加起来, 然后这条线是说

56:59 tokens from the past cannot communicate by setting them to negative infinity we're saying that we
将过去的东西设置为负无穷,无法传递

57:04 will not aggregate anything from those tokens and so basically this then goes through softmax and
无法从这些象征物中汇总任何信息, 因此基本上,这通过软体和软体

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

57:10 through the weighted and this is the aggregation through matrix multiplication and so what this is
通过加权, 这是通过矩阵乘法的汇总, 所以这是什么

matrix
矩阵 矩阵
数学与微积分

57:15 now is you can think of these as these zeros are currently just set by us to be zero but a quick
现在你可以想到这些了,因为现在我们只设定零是零,但很快

57:23 preview is that these affinities between the tokens are not going to be just constant at zero
预览是,当标牌之间的这些相近点不会在零时保持恒定

57:29 they're going to be data dependent these tokens are going to start looking at each other
数据上的数据依赖它们 这些符号将开始互相看对方

57:33 and some tokens will find other tokens more or less interesting and depending on what their values are
有些象征物会发现其他的象征物 多少或多或少有趣,取决于它们的价值是什么

57:39 they're going to find each other interesting to different amounts and i'm going to call those
他们会发现彼此有趣的不同金额 不同的金额,我称它们为

57:43 affinities i think and then here we are saying the future cannot communicate with the past
我想,然后我们在这里说 未来无法与过去交流

57:48 we're we're going to clamp them and then when we normalize and some we're going to aggregate
我们要去夹紧他们 然后当我们恢复正常时 和一些要聚集起来

57:54 a sort of their values depending on how interestingly find each other and so that's the preview for
取决于他们如何找到彼此 所以这是预览

57:59 self-attention and basically long story short from this entire section is that you can do weighted
自我注意,基本上长话短说 从整个部分的简短是 你可以做加权

attention
注意力 注意力
神经网络基础

58:06 aggregations of your past elements by having by using matrix multiplication of a lower triangular
使用下三角形矩阵乘法,将您过去元素的数组

matrix
矩阵 矩阵
数学与微积分

58:14 fashion and then the elements here in the lower triangular part are telling you how much of each
时时时时, 然后在下三角形部分的元素 显示每个元素有多少

58:19 element fuses into this position so we're going to use this trick now to develop the self-attention
元素的引信 进入这个位置 所以我们要用这个把戏 现在来开发自我注意

attention
注意力 注意力
神经网络基础

58:25 block so first let's get some quick preliminaries out of the way first the thing i'm kind of bothered
先找些快速的预科 来先让我有点烦恼

点积 · 向量 58:31

58:31 by is that you see how we're passing in vocab size into the constructor there's no need to do that
也就是你看到我们如何通过 瓦科卡的体积 进入建筑师,没必要这样做

58:35 because vocab size is already defined up top as a global variable so there's no need to pass this
因为 vocab 大小已被定义为上方的变量, 即是一个全球变量, 所以没有必要通过此选项

58:40 stuff around next what i want to do is i don't want to actually create i want to create
围绕接下来的事物,我想做的是 我不想真正创建我想要创建

58:46 like a level of indirection here where we don't directly go to the embedding for the
就像是间接程度 我们不直接去嵌入

> **embedding** [em'bediŋ]
> 嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
> 神经网络基础

58:51 logits but instead we go through this intermediate phase because we're going to start
登录,但相反我们通过中间阶段 因为我们要开始

> **logits**
> 对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
> 神经网络基础

58:55 making that bigger so let me introduce a new variable n embed it's short for number of
更大,所以让我介绍一个新的变量 n嵌入

59:01 embedding dimensions so n embed here will be say 32 that was the suggestion from github co-pilot
嵌入维度,所以嵌入到这里, 将意味着32, 这是Github 联合试点的建议

> **embedding** [em'bediŋ]
> 嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
> 神经网络基础

59:10 by the way um it also suggested 32 which is a good number so this is an embedding table and only
也建议32,这是一个不错的数字 所以这是个嵌入表

> **embedding** [em'bediŋ]
> 嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
> 神经网络基础

59:16 32 dimensional embeddings so then here this is not going to give us logits directly instead
32维嵌入器 所以这里不能直接给我们登入

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

59:23 this is going to give us token embeddings that's what i'm going to call it and then to go from the
给我们象征性的嵌入器 也就是我称之为它 然后从

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

59:27 token embeddings to the logits we're going to need a linear layer so self dot lm head let's call it
嵌入登录的符号, 我们需要一个线性层。 所以自点 lm 头让我们把它称为

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

59:33 short for language modeling head is an n linear from n embed up to vocab size and then when we
语言模拟头的短语言缩写为 n 线性, 从 n 嵌入到 vocab 大小, 然后当我们

59:40 swing over here we're actually going to get the logits by exactly what the co-pilot says
转过来 我们实际上要按照副驾驶员说的 来拿到登机记录

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

59:45 now we have to be careful here because this c and this c are not equal um this is an embed
现在我们必须小心,因为C和c不相等 嗯,这是嵌入

59:52 c and this is vocab size so let's just say that n embed is equal to c and then this just creates one
并且这是 vocab 大小, 所以让我们假设 n 嵌入等于 c , 然后这只创造一个

1:00:00 spurious layer of interaction through a linear layer but this should basically run so we see
线性层的相互作用 但是这基本上应该运行,这样我们可以看到

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:00:12 that this runs and uh this currently looks kind of spurious but uh we're going to build on top of
和呃,这个运行和 呃,目前这看起来似乎有点假冒 但是呃,我们将在

1:00:17 this now next up so far we've taken these in in the c's and we've encoded them based on the
至今为止我们已把这些在C的代码中, 我们根据这些编码

1:00:23 identity of the tokens inside idx the next thing that people very often do is that we're not just
代号内代号内标记的身份 之后人们经常做的就是 我们不只是

1:00:30 encoding the identity of these tokens but also their position so we're going to have a second position
将这些标记的身份编码, 并同时输入它们的位置, 所以我们将有一个第二个位置

1:00:36 embedding table here so self that position embedding table is an embedding of block size by
嵌入表格在此嵌入表格, 使位置嵌入表格的嵌入

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:00:42 n embed and so each position from zero to block size minus one will also get its own embedding
n 嵌入, 所以每个位置从零到块大小的区块大小为零再减 1 也将会有自己的嵌入

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:00:47 vector and then here first let me decode a b by t from idx that shape and then here we're also going to

然后这里的矢量 首先让我从形状的 idx 解码 b by t, 然后在这里我们也要

vector
向量 向量
数学与微积分

1:00:56 have a pause embedding which is the positional embedding and these are this is tordash arrange

存在一个暂停嵌入的嵌入,这是位置嵌入,这些是拖拉布式安排

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:01:01 so this will be basically just integers from zero to t minus one and all of those integers from

所以基本上只能从零到t的整数 减1和所有那些整数

1:01:07 zero to t minus one get embedded through the table to create a t by c and then here this gets

零到 t 减去 1 后嵌入表格以创建 c 的 t , 然后在这里

1:01:13 renamed to just say x and x will be the addition of the token embeddings with the positional embeddings

重新命名后仅表示 x和 x 将是用位置嵌入添加的象征性嵌入

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:01:21 and here the broadcasting note will work out so b by t by c plus t by c this gets right aligned

在这里,广播便条会解决的 所以b by t by c+ t by C 加上 t lt by c 这是正确的对齐

1:01:27 a new dimension of one gets added and it gets broadcasted across batch so at this point x holds

增加一个新维度的一个新维度,然后通过批量广播,这样此时x保持

dimension
维度 维度
数学与微积分

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:01:34 not just the token identities but the positions at which these tokens occur and this is currently
不仅象征性身份,而且这些象征性的发生位置,目前都是如此

1:01:40 not that useful because of course we just have a simple migraine model so it doesn't matter if you're
没用,因为当然我们只是有一个简单的偏头痛模型 所以不管你是不是

1:01:44 on the fifth position the second position or wherever it's all translation invariant at this stage
位于第五位置的第二位或目前翻译状态中

1:01:49 so this information currently wouldn't help but as we work on the self-attention block we'll see
所以目前这个信息不会有用 但是当我们研究自我注意区时 我们可以看到

attention
注意力 注意力
神经网络基础

1:01:54 that this starts to matter okay so now we get the crux of self-attention so this is probably
这一点开始重要 这没关系 所以现在我们得到自我注意的症结

attention
注意力 注意力
神经网络基础

1:02:02 the most important part of this video to understand we're going to implement a small
了解我们将要执行一个小

1:02:07 self-attention for a single individual head as they're called so we start off with where we were
一个人的头被称作"单人头", 自我自顾自顾, 所以我们从我们所在的地方开始

attention
注意力 注意力
神经网络基础

1:02:13 so all of this code is familiar so right now i'm working with an example where i change the number
所以所有代码都很熟悉 所以我现在正在用一个例子来修改号码

1:02:18 of channels from 2 to 32 so we have a 4 by 8 arrangement of tokens and each token and the
由2到32的频道,我们有4到8的 象征性和每个

1:02:25 information of each token is currently 32 dimensional but we just are working with random numbers
每个标记的信息目前为 32 维,但我们只是使用随机数字

1:02:31 now we saw here that the code as we had it before does a simple weight simple average
现在我们在这里看到,我们以前拥有的代码 做了一个简单的重量简单平均

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:02:38 of all the past tokens and the current token so it's just the previous information and current
过去的标记和当前标记中的所有过去标记的当前标记, 所以它只是以前的信息和当前

1:02:44 information is just being mixed together and in average and that's what this code currently
信息是混合在一起的,平均平均, 这就是这个代码目前

1:02:48 achieves and it does so by creating this lower triangular structure which allows us to mask out
建立这种低三角结构,使我们能够掩盖

1:02:54 this weight matrix that we create so we mask it out and then we normalize it and currently
重力矩阵,我们创造的这个重量矩阵, 我们制造的, 所以我们把它遮盖出来 然后我们把它正常化

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

matrix
矩阵 矩阵
数学与微积分

1:03:02 when we initialize the affinities between all the different sort of tokens or nodes i'm going
当我们开始在所有不同种类的象征物或节点之间 建立亲近关系时,我要去

1:03:08 use those terms interchangeably so when we initialize the affinities between all the different
当我们开始使用所有不同对象之间的亲和

1:03:13 tokens to be zero then we see that way gives us this structure where every single row has these
将值为零,然后我们可以看到这样的方式给我们这个结构,每个行都有这样的结构

1:03:21 uniform numbers and so that's what that's what then in this matrix multiply makes it so that
统一数字,所以这就是这个矩阵中 乘以的值

matrix
矩阵 矩阵
数学与微积分

1:03:27 we're doing a simple average now we don't actually want this to be all uniform because
我们做一个简单的平均,现在我们不希望 这一切都是统一的,因为

1:03:35 different tokens will find different other tokens more or less interesting and we want that to be
不同牌子将发现不同其他牌子或多或少有趣的不同牌子,我们希望它

1:03:40 data dependent so for example if i'm a vowel then maybe i'm looking for consonants in my past
数据依赖数据,例如,如果我是元音 那么也许我在找过去时的拼音

1:03:46 and maybe i want to know what those consonants are and i want that information to flow to me
也许我想知道这些音调是什么 我想让这些信息 传到我

1:03:51 and so i want to now gather information from the past but i want to do it in a data dependent
所以我现在想收集过去的信息 但我想用数据来做

1:03:55 way and this is the problem that self- attention solves now the way self- attention solves this
这也是自我自用解决的问题 现在自用解决的方法

attention
注意力 注意力
神经网络基础

1:04:01 is the following every single node or every single token at each position will emit two vectors
即下面每个节点或每个位置的每个标记将释放两个矢量。

1:04:08 it will emit a query and it will emit a key now the query vector roughly speaking is what am i looking
它会发出查询, 并会发出一个关键键 现在的查询矢量大致表达是 我所看到的

vector
向量 向量
数学与微积分

1:04:17 for and the key vector roughly speaking is what do i contain and then the way we get
和关键矢量 粗略的表达方式是 我包含什么 然后我们得到的方式

vector
向量 向量
数学与微积分

1:04:23 affinities between these tokens now in a sequence is we basically just do a dot product
这些象征物之间的相似性 正在按顺序排列 我们基本上只是做点产品

dot product
点积 点积
数学与微积分

1:04:30 between the keys and the queries so my query dot products with all the keys of all the other tokens
键和查询之间,所以我的查询点产品与所有其他符号的键

1:04:37 and that dot product now becomes way and so if the key and the query are sort of aligned
点产品现在变成了方向,所以如果键和查询是某种对齐

dot product
点积 点积
数学与微积分

1:04:46 they will interact to a very high amount and then i will get to learn more about that specific
之后我就会学到更多关于这个具体

1:04:52 token as opposed to any other token in the sequence so let's implement this now we're going
而不是序列中的其他任何符号, 所以让我们执行它吧, 现在我们要开始

1:05:01 to implement a single what's called head of self- attention so this is just one head there's a hyper
来实施一个所谓的自我自省头 所以这只是一个头

attention
注意力 注意力
神经网络基础

1:05:10 parameter involved these heads which is the head size and then here i'm initializing linear modules
参数包含这些头部, 即头大小, 然后在这里我初始化线性模块

parameter [pə'ræmitə]
参数 机器学习系统自行训练的模型的变量。例如，权重 就是一种参数，它们的值是机器 学习系统通过连续的训练迭代逐渐学习到的。与超参数 相对。
神经网络基础

1:05:16 and i'm using biasing equals false so these are just going to apply matrix multiply with some fixed
我用的是假的偏见,所以这些只是应用矩阵乘法 和一些固定的

matrix
矩阵 矩阵
数学与微积分

1:05:21 weights and now let me produce a key and q k and q by forwarding these modules on x
现在让我用x 传送这些模块来生成一个键和qk k 与qq ,

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:05:29 so the size of this will now become b by t by 16 because that is the head size and the same here
所以此大小将变成 b 乘 t 乘 16 , 因为是头大小和此处相同

1:05:37 b by t by 16 that's being the head size so you see here that when i forward this linear on top of my
16T16,这就是头大小 所以在这里你可以看到 当我把线性 放在我上面

1:05:49 x all the tokens in all the positions in the b by t arrangement all of them in parallel and
x 以 t 排列 B 中所有位置的所有符号, 全部同时并列

1:05:55 independently produce a key and a query so no communication has happened yet but the
独立生成一个键和查询,所以尚未发生通信,但

1:06:02 communication comes now all the queries will dot product with all the keys
所有的查询都用所有钥匙 来做点产品

dot product
点积 点积
数学与微积分

1:06:08 so basically what we want is we want the way now or the affinities between these to be
所以基本上我们想要的就是 我们希望现在的方式 或者两者之间的亲和

1:06:13 query multiplying key but we have to be careful with uh we can't matrix multiply this we actually
查询乘法键 但是我们必须小心,呃 我们不能用矩阵来乘法

matrix
矩阵 矩阵
数学与微积分

1:06:19 need to transpose uh k but we have to be also careful because these are we need to have
需要转换 uh k 但是我们也要小心 因为这些都是我们需要的

1:06:25 the bash dimension so in particular we want to transpose uh the last two dimensions dimension
以特别的,我们想转换 最后两个维度

dimension
维度 维度
数学与微积分

1:06:31 negative one and dimension negative two so negative two negative one and so this matrix

dimension
维度 维度
数学与微积分

matrix
矩阵 矩阵
数学与微积分

1:06:38 multiply now will basically do the following b by t by 16 matrix multiplies b by 16 by t
乘以 16 矩阵乘以 b乘16 t

matrix
矩阵 矩阵
数学与微积分

1:06:48 to give us b by t by t so for every row of b we're now going to have a t square matrix
给我们一两两行的Bb 来给一两两,然后我们再有一个t方形矩阵

matrix
矩阵 矩阵
数学与微积分

1:07:00 giving us the affinities and these are now the way so they're not zeros they are now coming from
给我们亲和关系,现在这些是正路 所以它们不是零

1:07:06 this dot product between the keys and the queries so this can now run i can i can run this
此在密钥和查询之间的点产品, 这样它现在可以运行了 我可以运行这个

dot product
点积 点积
数学与微积分

1:07:13 and the weighted aggregation now is a function in a data band and manner
加权总和现在是一个数据带和方式的函数

1:07:17 between the keys and queries of these nodes so just inspecting what happened here the way
和这些节点的密钥和查询之间, 所以只要检查这里发生的事情

1:07:24 takes on this form and you see that before way was just a constant so it was applied in the same way
以这个形式在这种形式上, 你可以看到前方只是一个常数, 所以它应用的方式是一样的

1:07:31 to all the batch elements but now every single batch elements will have different sort of way
对所有批量元素的组装成分,但现在每个批量元素将采用不同的方式

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:07:36 because uh every single batch element contains different tokens at different positions and so this
因为每个批量元素 都包含不同位置的不同牌子 所以这个

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:07:42 is now data dependent so when we look at just the zeroth uh row for example in the input these are the
数据现在取决于数据,所以当我们只看零行时, 例如输入中的零行就是

1:07:49 weights that came out and so you can see now that they're not just exactly uniform um and in particular
的重量,出来,所以你可以看到 现在它们不完全 完全一样,嗯,特别是

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:07:55 as an example here for the last row this was the eighth token and the eighth token knows what
举最后一个行的例子,这里的最后一行 这是第八代,第八代知道什么

1:08:00 content it has and it knows at what position it's in and now the eighth token based on that
并且知道它的位置 以及现在基于它的第八代

1:08:07 uh creates a query hey i'm looking for this kind of stuff um i'm a vowel i'm on the eighth position
来个问询 嘿 我正在找这种东西 我是元音 我第八位

1:08:12 i'm looking for any consonants at positions up to four and then all the nodes get to emit keys
我在找四级以下位置的任何音符 然后所有节点都发出键

1:08:19 and maybe one of the channels could be i am a i am a consonant and i am in a position up to four
也许其中一种渠道 可能是我是一个和音的调音 我最多能坐4个

1:08:25 and that key would have a high number in that specific channel and that's how the query and
关键在那个特定的频道里 会有很多数字 这就是问题和

1:08:30 the key when they dot product they can find each other and create a high affinity and when they
当他们点出产品时,当他们互相发现,创造至亲的时候,当他们发现的时候,

dot product
点积 点积
数学与微积分

1:08:34 have a high affinity like say this token was pretty interesting to uh to this eighth token
和高亲近,比如说这个象征物 对这八种象征物来说很有趣

1:08:42 when they have a high affinity then through the softmax i will end up aggregating a lot of its
当他们有高度的亲和关系 然后通过软式的I 最终会聚集很多它

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:08:46 information into my position and so i'll get to learn a lot about it now just this we're looking
信息进入我的位置 所以我现在可以学到很多关于它的信息了

1:08:54 at way after this has already happened um let me erase this operation as well so let me erase the
这件事已经发生了之后... 让我也抹掉这个行动

operation
运算 运算
神经网络基础

1:09:01 masking and the softmax just to show you the under the hood internals and how that works
面具和软顶 只是为了让你看看 兜帽下的内部,以及它是如何运作的

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:09:06 so without the masking and the softmax way it comes out like this right this is the outputs of the
所以没有面具和软式的 方式,它出来像这样的权利 这是输出

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:09:11 dot products um and these are the raw outputs and they take on values from negative you know
这些都是原始产出,它们吸收了负值的数值

1:09:16 two to positive two etc so that's the raw interactions and raw affinities between all the
两者的原始相互作用和原始亲近性

1:09:23 nodes but now if i'm a if i'm a fifth node i will not want to aggregate anything from the
结点 但现在 如果我是第五个结点的话

1:09:28 sixth node seventh node and the eighth node so actually we use the upper triangular masking so
第六节点七节节点和第八节点,所以我们实际上使用上三角形遮罩

1:09:34 those are not allowed to communicate and now we actually want to have a nice uh distribution
不允许交流 而现在我们其实想要有一个不错的uh分发

distribution
分布 分布
数学与微积分

1:09:42 so we don't want to aggregate negative point one one of this node that's crazy so instead we
所以我们不想把一个结点中的负点 合并到一个结点上,这太疯狂了 所以我们

1:09:47 exponentiate and normalize and now we get a nice distribution that sums to one and this
和正常,现在我们得到一个很好的分布 将一个和这个

distribution
分布 分布
数学与微积分

1:09:51 is telling us now in the data dependent manner how much of information to aggregate from any
现在,我们以数据依赖的方式告诉我们,从任何来源收集的信息有多少可以汇总。

1:09:56 of these tokens in the past so that's way and it's not zeros anymore but but it's calculated in this
过去这些标志品的过去,所以这是方式 和它不是零了,但 它计算在这个

1:10:04 way now there's one more uh part to a single self-attention head and that is that when we
单一个自知头 还有另一个... ...

attention
注意力 注意力
神经网络基础

1:10:10 do the aggregation we don't actually aggregate the tokens exactly we aggregate we produce one
做总和,我们实际上不实际加集 符号,我们准确加集 我们生产一个

1:10:16 more value here and we call that uh the value so in the same way that we produced
更多的价值在这里,我们称之为 呃这个价值 所以以同样的方式 我们生产

1:10:22 pm query we're also going to create a value and then here we don't aggregate x we calculate a v
pp问 pp 查询,我们也会创建一个值, 然后在这里我们不汇总 x 我们计算一个 v

1:10:33 which is just achieved by uh propagating this linear on top of x again and then we
仅仅通过在x的顶部 传播这个线性 然后我们

1:10:40 output way multiplied by v so v is the elements that we aggregate or the the vector that we
输出输出乘 v so v 是我们加起来的元素或向量

vector
向量 向量
数学与微积分

1:10:46 aggregate instead of the raw x and now of course uh this will make it so that the output here of the
而不是原x,现在当然是... 这样就可以让它的输出

1:10:53 single head will be 16 dimensional because that is the head size so you can think of x as kind of
单头的一头将是一个16维的大小, 因为是头的大小, 所以您可以把 X 视为

1:10:59 like private information to this token if you if you think about it that way so x is kind of
如果您这样想, x 是一种

1:11:04 private to this token so i'm a fifth token at some and i have some identity and uh my information
私人的这个标志 所以我是第五个标志 在一些人和我有 一些身份和呃 我的信息

1:11:11 is captain vector x and now for the purposes of the single head here's what i'm interested in here's
向量x 船长,现在为了一个头的目的... ...这里是我感兴趣的

vector
向量 向量
数学与微积分

1:11:17 what i have and if you find me interesting here's what i will communicate to you and that's stored
我有什么,如果你发现我有趣... ...我会告诉你的,然后储存

1:11:23 in v and so v is the thing that gets aggregated for the purposes of this single head between the
和此介于

1:11:29 different notes and that's uh basically the self- attention mechanism this is this is what it
不同的笔记,嗯,基本上这个是自注意机制 这就是它是什么

attention
注意力 注意力
神经网络基础

1:11:36 does there are a few notes that i would make like to make about attention number one attention is a

是否有我愿意指出的一点,我想指出的是,首先注意的是

attention
注意力 注意力
神经网络基础

卷积 · 批次大小 1:11:43

1:11:43 communication mechanism you can really think about it as a communication mechanism where you

将它视为一种交流机制,

1:11:48 have a number of nodes in a directed graph where basically you have edges pointed between nodes

在一个定向图形中有一些节点, 其中基本上您在节点之间有边缘

1:11:52 like this and what happens is every node has some vector of information and it gets to

像这样,然后发生的事情 每个节点都有一定的信息矢量 它可以到达

vector
向量 向量
数学与微积分

1:11:58 aggregate information via a weighted sum from all of the nodes that point to it and this

以所有指及其所在节点的加权和

1:12:05 is done in a data dependent manner so depending on whatever data is actually stored at each node

数据依据数据进行,取决于每个节点实际储存的任何数据。

1:12:09 at any point in time now our graph doesn't look like this our graph has a different structure

任何时刻 我们的图表看起来都不像这个样子 我们的图形结构不同

1:12:15 we have eight nodes because the block size is eight and there's always eight tokens and the first

我们有八个节点 因为区块大小是八个 总是有八个纪念品 第一个

1:12:22 node is only pointed to by itself the second node is pointed to by the first node and itself

节点本身仅指第一个节点和它本身指的第二个节点

1:12:27 all the way up to the eighth node which is pointed to by all the previous nodes and itself

直至第八节点,前所有节点和其本身都指出了这个节点。

1:12:33 and so that's the structure that our directed graph has or happens happens to have an
这就是我们指示的图表 是否有或碰巧有

1:12:37 autoregressive sort of scenario like language modeling but in principle attention can be applied
语言建模等自动递减情景,但原则上可以予以注意

attention
注意力 注意力
神经网络基础

1:12:42 to any arbitrary directed graph and it's just a communication mechanism between the nodes
任意直向图形,只是节点之间的通讯机制

1:12:47 the second note is that note is that there's no notion of space so attention simply acts
第二注是注意的是,没有空间的概念 所以注意只是行为

attention
注意力 注意力
神经网络基础

1:12:52 over like a set of vectors in this graph and so by default these nodes have no idea where they
类似此图形中的一组矢量, 所以默认情况下这些节点不知道它们在哪里

1:12:57 are positioned in the space and that's why we need to encode them positionally and
所以我们需要用定位编码

1:13:02 sort of give them some information that is anchored to a specific position so that they
给他们一些信息 以特定位置为基点,以便他们

1:13:06 sort of know where they are and this is different than for example from convolution because if you
和例如与变化不同 因为如果你

convolution [.kɒnvə'lu:ʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

1:13:11 run for example a convolution operation over some input there is a very specific sort of layout
例如,运行 , 某输入的混和操作, 其布局非常具体

convolution [.kɒnvə'lu:ʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

operation
运算 运算
神经网络基础

1:13:16 of the information in space and the convolutional filters sort of act in space and so it's
空间信息 和革命过滤器 在空间的行为,所以它

1:13:22 not like in attention in attention is just a set of vectors out there in space they communicate
不像关注中的注意 仅仅是空间中一系列矢量 它们交流的

attention
注意力 注意力
神经网络基础

1:13:28 and if you want them to have a notion of space you need to specifically add it which is what we've
如果你想让他们有空间的概念 你需要特别添加它 这正是我们所拥有的

1:13:33 done when we calculated the relative the positional encode encodings and added that information to the
当我们计算位置编码编码的相对编码, 并将该信息添加到

1:13:39 vectors the next thing that I hope is very clear is that the elements across the batch dimension
接下来我希望非常清楚的是 分批量的元素

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

dimension
维度 维度
数学与微积分

1:13:44 which are independent examples never talk to each other they're always processed independently
独立的例子 永远不互相交谈 他们总是独立处理

1:13:48 and this is a bashed matrix multiply that applies basically a matrix multiplication
这是一个被击碎的矩阵乘法, 基本上应用矩阵乘法

matrix
矩阵 矩阵
数学与微积分

1:13:52 kind of in parallel across the batch dimension so maybe it would be more accurate to say that
批量尺寸之间平行的类似, 所以说这个也许更准确

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

dimension
维度 维度
数学与微积分

1:13:57 in this analogy of a directed graph we really have because the batch size is four we really have
类比我们真正有的定向图表 因为批量大小是4

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:14:02 four separate pools of eight nodes and those eight nodes only talk to each other but in total
由八个节点组成的四个单独的集合群,这八个节点只相互交谈,但总数

1:14:07 there's like 32 nodes that are being processed but there's sort of four separate pools of eight
大约32个节点正在处理中 但有4个不同的8个池

1:14:13 you can look at it that way the next note is that here in the case of language modeling
您可以这样看它, 下注是这里的语种模拟

1:14:19 we have this specific structure of directed graph where the future tokens will not
我们拥有这个特定图示结构, 指示图显示未来标牌不会

1:14:24 communicate to the past tokens but this doesn't essentially have to be the constraint in the
发送到过去标记的邮件, 但基本上这并不一定是

1:14:29 general case and in fact in many cases you may want to have all of the nodes talk to each other
通常情况下,事实上在许多情况下 你们可能想要让所有节点 相互交谈

1:14:35 fully so as an example if you're doing sentiment analysis or something like that with a transformer
完全的,作为一个例子 如果你做情绪分析 或者类似的东西

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:14:40 you might have a number of tokens and you may want to have them all talk to each other fully
可能有很多的标志,也许你们想让它们彼此充分交谈

1:14:45 because later you are predicting for example the sentiment of the sentence and so it's okay
因为后来你预测了 例如判刑的情绪 所以没关系

1:14:49 for these nodes to talk to each other and so in those cases you will use an encoder block of
以便这些节点相互交谈 所以在这种情况下 你会用一个编码器块

1:14:55 self- attention and all it means that it's an encoder block is that you will delete this line of code
自我注意,它意味着 它是一个编码器块, 就是您要删除此代码行

attention
注意力 注意力
神经网络基础

1:15:01 allowing all the nodes to completely talk to each other what we're implementing here is sometimes
让所有节点 能够完全地相互交谈 我们在这里执行的是有时

1:15:06 called a decoder block and it's called a decoder because it is sort of like a decoding language
叫做解码器块,它叫解码器 因为有点像解码语言

1:15:14 and it's got this autoregressive format where you have to mask with the triangle and matrix
它有一种自动递后格式, 你必须用三角形和矩阵掩罩

matrix
矩阵 矩阵
数学与微积分

1:15:19 so that nodes from the future never talk to the past because they would give away the answer
以便未来之结点永不与过去对话,因为他们会放弃答案

1:15:25 and so basically in encoder blocks you would delete this allow all the nodes to talk
所以基本上在编码器块中, 您将会删除此选项让所有节点都能够交谈

1:15:29 in decoder blocks this will always be present so that you have this triangular structure
的解码符块中, 此将总是存在, 这样您就可以使用此三角形结构

1:15:34 but both are allowed and attention doesn't care attention supports arbitrary connectivity
两者皆允许,但关注并不关心 支持任意连通

attention
注意力 注意力
神经网络基础

1:15:38 between nodes the next thing I wanted to comment on is you keep me you keep hearing me say
接在节点之间,接下来我想评论的是 你让我一直你 不断听到我说

1:15:43 attention self- attention etc there's actually also something called cross attention what is the
注意自我关注等 事实上还有某种东西 需要交叉关注

attention
注意力 注意力
神经网络基础

1:15:47 difference so basically the reason this attention is self- attention is because the keys queries and
所以基本上区别 所以,这种关注的 原因自觉的注意 因为关键询问和

attention
注意力 注意力
神经网络基础

1:15:56 the values are all coming from the same source from x so the same source x produces keys queries
数值都来自同一来源的相同源代码 x,所以同一个源源 x 产生关键查询

1:16:03 and values so these nodes are self-attending but in principle attention is much more general
和价值,所以这些节点是自命自职的,但原则上注意的范围要广得多。

attention
注意力 注意力
神经网络基础

1:16:09 than that so for example in encoder decoder transformers you can have a case where the
可以在一个情况下

1:16:14 queries are produced from x but the keys and the values come from a whole separate external source
x 的查询由x 生成,但键和值来自一个单独的外部源

1:16:19 and sometimes from encoder blocks that encode some context that we'd like to condition on
有时从编码器块中 编码了我们想以

1:16:25 and so the keys and the values will actually come from a whole separate source those are
所以键和值实际上来自一个 完全独立的源

1:16:29 nodes on the side and here we're just producing queries and we're reading off information from
一边的节点和这里 我们只是提出询问 而我们读取的信息

1:16:33 the side so cross attention is used when there's a separate source of nodes we'd like to pull
侧侧,所以交叉关注被使用 当有单独的节点源时 我们想拉

attention
注意力 注意力
神经网络基础

1:16:41 information from into our nodes and it's self-attention if we just have nodes that would like to look
输入节点的信息,如果 我们只有希望查看的节点, 这也是自我注意

attention
注意力 注意力
神经网络基础

1:16:46 at each other and talk to each other so this attention here happens to be self-attention
彼此互相交谈,相互交谈 所以这里的注意力 恰好是自顾自顾

attention
注意力 注意力
神经网络基础

1:16:52 but in principle attention is a lot more general okay and the last note at this stage is if we
但原则上注意的原则上是 更普遍地说,我们是否同意

attention
注意力 注意力
神经网络基础

1:16:59 come to the attention is all you need paper here we've already implemented attention so given
注意的是这里需要的文件 我们已经把注意力都投入了

attention
注意力 注意力
神经网络基础

学习速率 · 卷积 1:17:03

1:17:03 query key and value we've multiplied the query and the key we've softmaxed it and then we
查询键和值 我们已经将查询和关键键乘以 我们软化了它 然后我们

1:17:09 are aggregating the values there's one more thing that we're missing here which is the dividing by
集合了这些价值 还有一件事我们在这里缺少的是 那就是分除

1:17:13 one over square root of the head size the decay here is the head size why are they doing this one
在一个平方根上,头大小的一对一 腐烂在这里是头大小 他们为什么这样做

1:17:19 is this important so they call it a scaled attention and it's kind of like an important
很重要,所以他们称之为 扩大关注和它 就像一个重要

attention
注意力 注意力
神经网络基础

1:17:25 normalization to basically have the problem is if you have unit Gaussian inputs so zero mean
基本正常化的问题在于 如果你有单位高西亚输入,那么零平均

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

1:17:30 unit variance k and q are unit Gaussian then if you just do way naively then you see that
单位单位差 k和q 是单位单位高西亚, 如果你只是天真地做的话, 你就会看到

1:17:36 your way actually will be the variance will be on the order of head size which in our case is 16
以你的方式,实际上 差异将按头大小的顺序 以我们的情况来说是16

1:17:42 but if you multiply by one over head size square root so this is square root and this is one over
但如果您乘以一个大于头的平方根,则此为平方根。 这是平方根 , 此为平方正根

1:17:48 then the variance of way will be one so it will be preserved now why is this important
道路的差别就不同了,所以现在应该保留下来 为什么这很重要

1:17:54 you'll notice that way here will feed into softmax and so it's really important especially at
你会注意到这里的这种方式 将进软峰体,所以它非常重要

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:18:01 initialization that way be fairly diffuse so in our case here we sort of locked out here and way
初始化的方式相当分散 所以在这里我们这里 我们被锁在了这里和路上

1:18:09 had a fairly diffuse numbers here so like this now the problem is that because of softmax if way
这里的号码分布相当分散 现在的问题是 因为如果是软的

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:18:16 takes on very positive and very negative numbers inside it softmax will actually converge towards
软峰内非常正的负数 和非常正的反数 将实际上接近

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:18:22 one hot vectors and so I can illustrate that here say we are applying softmax to a
一种热矢量,所以我可以说明 我们用软通

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:18:30 tensor of values that are very close to zero then we're gonna get a diffuse thing out of softmax
离零差的值非常接近于0 然后我们就会从软体中 得到一个扩散的东西

tensor ['tensə]
张量 TensorFlow 程序中的主要数据结构。张量是 N 维（其中 N 可能非常大）数据结构，最常见 的是标量、向量或矩阵。张量的元素可以包含整数值、浮点值或字符串值。
数学与微积分

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:18:35 but the moment I take the exact same thing and I start sharpening it making it bigger by multiplying
但当我采取同样的事情 开始磨它 通过乘法来使它变得更大

1:18:39 these numbers by eight for example you'll see that the softmax will start to sharpen and in fact it
这些数字乘以8乘以8,例如你会看到软峰开始变亮,事实上它会变亮

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:18:44 will sharpen towards the max so it will sharpen towards whatever number here is the highest
将达到最高点,以达到最高点 从而达到最高点

1:18:50 and so basically we don't want these values to be too extreme especially at initialization
所以基本上我们不希望这些价值 过于极端,特别是在初始化时

1:18:54 otherwise softmax will be way too peaky and you're basically aggregating information from like a single
否则软体的峰值会太高 基本上你正在收集信息 就像一个单

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:19:01 node every node just aggregates information from a single other node that's not what we want especially
每一个节点都只是汇总其他一个节点的信息, 这不是我们特别想要的

1:19:05 at initialization and so the scaling is used just to control the variance at initialization
缩放用于控制初始化时的差异

1:19:11 okay so having said all that let's now take our self-attention knowledge and let's take it for a spin
所以说完所有的话 让我们现在用我们自知知识 来兜一圈

attention
注意力 注意力
神经网络基础

1:19:18 so here in the code I've created this head module and implements a single head of self-attention
所以在代码中,我创建了这个头模块 并用一个自控的单头

attention
注意力 注意力
神经网络基础

1:19:23 so you give it the head size and then here it creates the key query and the value linear layers
,然后给它头大小, 然后在这里创建关键查询和线性值层

1:19:29 typically people don't use biases in these so those are the linear projections that we're going
通常人们不会使用偏见,所以这些是线性预测 我们要去的

1:19:33 to apply to all of our nodes now here I'm creating this trill variable trill is not a
来应用到我们所有的节点, 现在在这里我正在创建这个三轮变量曲项

1:19:39 parameter of the module so in sort of pytorch naming conventions this is called a buffer
模块参数参数的参数,在类似 Pytorch 命名常规中,这被称为缓冲

parameter [pə'ræmitə]
参数 机器学习系统自行训练的模型的变量。例如，权重 就是一种参数，它们的值是机器 学习系统通过连续的训练迭代逐渐学习到的。与超参数 相对。
神经网络基础

pytorch
PyTorch PyTorch
代码与工程

1:19:44 it's not a parameter and you have to call it you have to assign it to the module using a
它不是一个参数, 您必须把它称为您必须使用 a 的

parameter [pə'ræmitə]
参数 机器学习系统自行训练的模型的变量。例如，权重 就是一种参数，它们的值是机器 学习系统通过连续的训练迭代逐渐学习到的。与超参数 相对。
神经网络基础

1:19:48 registered buffer so that creates the trill the trying lower triangular matrix and when we're given
注册缓冲键, 这样创建三角形的三角式下方试样矩阵和当我们被授予时

matrix
矩阵 矩阵
数学与微积分

1:19:54 the input x this should look very familiar now we calculate the keys the queries we can only
输入 x x 输入后, 此输入会看起来非常熟悉, 现在我们计算出我们只能查询的密钥

1:19:59 clock it the attention scores inside way we normalize it so we're using scaled attention here
注意在内部计时 我们把它正常化,所以我们在这里使用 扩大的注意力

attention
注意力 注意力
神经网络基础

1:20:06 then we make sure that sure doesn't communicate with the past so this makes it a decoder block
然后我们确保不与过去交流 所以这使它成为一个解码器块

1:20:11 and then softmax and then aggregate the value and output
并随后将数值和输出相加

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:20:15 then here in the language model I'm creating a head in the constructor and I'm calling it self-attention

而在语言模型中,我正在制造一个建筑商的头部, 我称之为自我关注

attention
注意力 注意力
神经网络基础

1:20:21 head and the head size I'm going to keep as the same and embed just for now and then here

头和头大小,我打算保持不变 并嵌入只是暂时的

1:20:29 once we've encoded the information with the token embeddings and the position embeddings

一旦我们编码了信息 与象征性嵌入和位置嵌入

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:20:34 we're simply going to feed it into the self-attentioned head and then the output of that is going to

我们只是把它注入到 自我意识的头部 然后它的输出就会

1:20:38 go into the decoder language modeling head and create the logits so this is sort of the simplest

进入解码器语言模拟头,创建登录,这样就简单多了

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:20:46 way to plug in a self-attention component into our network right now I had to make one more change

插上一个自我注意组件的方法 现在插进我们的网络中 现在我不得不再做一次改变

attention
注意力 注意力
神经网络基础

1:20:52 which is that here in the generate we have to make sure that our idx that we feed into the

也就是在产品中 我们必须确保我们的IDx

1:20:59 model because now we're using positional embeddings we can never have more than block size
模型,因为我们现在使用定位嵌入器 我们的区块大小是无法超过的

embedding [em'bedɪŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:21:05 coming in because if idx is more than block size then our position embedding table is going to
是因为如果 IDx 超过区块大小,那么我们的位置嵌入表格将会

embedding [em'bedɪŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:21:10 run out of scope because it only has embeddings for up to block size and so therefore I added some
范围已超出,因为只有嵌入的尺寸最多为块的块大小,所以我加了一些

embedding [em'bedɪŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:21:15 code here to crop the context that we're going to feed into self so that we never pass in more
代码在这里裁剪下我们将要输入到自我的背景, 这样我们再也不会通过更多

1:21:23 than block size elements so those are the changes and let's now train the network
区域大小元素, 所以这些是变化, 现在让我们训练网络

1:21:27 okay so I also came up to the script here and I decreased the learning rate because
所以我也来到了这里的剧本 我降低了学习率 因为

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:21:32 the self- attention can't tolerate very very high learning rates and then I also increased
自我自关心不能容忍非常高的学习率 然后我也提高了

attention
注意力 注意力
神经网络基础

1:21:36 number of iterations because the learning rate is lower and then I trained it and previously we were
迭代次数,因为学习率较低,然后我培训了它,以前我们

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:21:41 only able to get to up to 2.5 and now we are down to 2.4 so we definitely see a little bit of
只能达到2.5到2.5 现在我们降到2.4了 所以我们绝对可以看到

1:21:46 improvement from 2.5 to 2.4 roughly but the text is still not amazing so clearly the
大约从2.5改善为2.4,但案文仍然不令人惊叹,因此十分清楚

1:21:52 self- attention head is doing some useful communication but we still have a long way to go
自我自觉的头部正在做一些有用的交流 但我们还有很长的路要走

attention
注意力 注意力
神经网络基础

1:21:59 okay so now we've implemented the scale.product attention now next up and the attention is all
产品关注,现在又开始关注

attention
注意力 注意力
神经网络基础

1:22:03 you need paper there's something called multi-head attention and what is multi-head attention it's
需要纸,你需要纸 有种东西叫做多头人关注

attention
注意力 注意力
神经网络基础

1:22:09 just applying multiple attentions in parallel and concatenating the results so they have a little
将多重关注平行地运用到一起 并且把结果集中起来 这样他们就能有一点点

1:22:14 bit of diagram here I don't know if this is super clear it's really just multiple attentions
这张图的位数 我不知道是否非常清楚 这真的只是多点关注而已

1:22:20 in parallel so let's implement that fairly straightforward if we want a multi-head
并同时执行,所以如果我们想要一个多头人的话,让我们实施这个简单明了的

1:22:26 attention then we want multiple heads of self-attention running in parallel so in PyTorch we can do this

然后我们关注,然后我们想要多位自我自控的负责人 平行运行,所以在皮托尔奇 我们可以做到这一点

attention
注意力 注意力
神经网络基础

pytorch
PyTorch PyTorch
代码与工程

1:22:32 by simply creating multiple heads so however many heads you want and then what is the head size of

仅仅通过创建多个头部, 如此多的头部,无论你想要多少个头, 然后什么是头大小

1:22:40 each and then we run all of them in parallel into a list and simply concatenate all of the

然后我们把所有的 都平行地 一起排成一个清单 简单地把所有

1:22:47 outputs and we're concatenating over the channel dimension so the way this looks now is we don't

输出结果,我们正在对频道的维度进行分解 所以现在看来 我们没有

dimension
维度 维度
数学与微积分

1:22:53 have just a single attention that has a head size of 32 because remember an embed is 32 instead of

只需注意一个单项,头的尺寸为32, 因为记住嵌入是32而不是32

attention
注意力 注意力
神经网络基础

1:23:01 having one communication channel we now have four communication channels in parallel and each one of

现有四个平行通信渠道,每个

1:23:08 these communication channels typically will be smaller correspondingly so because we have four

这些交流渠道通常会相应缩小,因为我们有四个

1:23:15 communication channels we want eight-dimensional self-attention and so from each communication

我们希望八维自我注意 等从每条通讯中

attention
注意力 注意力
神经网络基础

1:23:20 channel we're going to gather eight-dimensional vectors and then we have four of them and that
频道,我们收集八维矢量 然后我们有四个

1:23:25 concatenates to give us 32 which is the original and embed and so this is kind of similar to if
给我们32个原始和嵌入的 原封和嵌入,所以这有点类似

1:23:31 you're familiar with convolutions this is kind of like a group convolution because basically
你熟悉革命,这有点像团体革命 因为基本上

convolution [.kɒnvə'lu:ʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

1:23:35 instead of having one large convolution we do convolution in groups and that's multi-headed
我们不是要一个大革命 而是要组成一个集团, 并且这个多头的

convolution [.kɒnvə'lu:ʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

1:23:40 self- attention and so then here we just use SA heads self- attention heads instead
自我自留 所以在这里 我们只用SA头的自留头来代替

attention
注意力 注意力
神经网络基础

1:23:47 now I actually ran it and scrolling down I ran the same thing and then we now get this down to
现在我实际上运行了它,滚下来 滚下来,我运行了同样的事情 然后我们得到这个

1:23:54 2.28 roughly and the output is still the generation is still not amazing but clearly the validation
2.28 大致上说来,产出仍然是产生出来的,但显然,这还不是惊人的,而是明显的验证。

validation
验证 验证
训练与优化

1:24:00 loss is improving because we were at 2.4 just now and so it helps to have multiple communication
损失在不断改善,因为我们刚刚2.4岁 所以多沟通会帮助我们

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:24:06 channels because obviously these tokens have a lot to talk about they want to find the
因为这些标记显然有很多要谈论的话题, 他们想要找到

1:24:11 consonants the vowels they want to find the vowels just from certain positions they want to
和他们想要找到的元音一致 他们想从某些位置

1:24:15 find any kinds of different things and so it helps to create multiple independent channels of
找到任何种类的不同的东西 所以它有助于创造多种独立的渠道

1:24:20 communication gather lots of different types of data and then decode the output now going back
通信通信收集大量不同类型数据,然后解码输出,现在返回

1:24:26 to the paper for a second of course I didn't explain this figure in full detail but we are
当然,我还没有详细解释这个数字 但我们

反向传播 · 梯度 1:24:30

1:24:30 starting to see some components of what we've already implemented we have the positional
开始看到一些我们已经实施的部分组成部分 我们已经有了位置

1:24:33 encodings the token encodings that add we have the masked multi-headed attention
编码添加的代号编码,我们得到了蒙面多头关注

attention
注意力 注意力
神经网络基础

1:24:38 implemented now here's another multi-headed attention which is a cross- attention to
这也是另一个多头关注的焦点,

attention
注意力 注意力
神经网络基础

1:24:43 an encoder which we haven't we're not going to implement in this case I'm going to come back to
我们没有的编码器,我们不会执行 在本案中,我将要回来

1:24:47 that later but I want you to notice that there's a feet forward part here and then this is grouped
稍后,但我要你注意 这里有一英尺向前 然后这个分组

1:24:53 into a block that gets repeated to get in again now the feet forward part here is just a simple
进入一个街区, 重复得到 再次进入 现在脚向前的 部分在这里只是一个简单的

1:24:58 multi- layer presumptron so the multi-headed so here position wise feet forward networks
多层预发式天文仪,所以多头人因此在这里定位 明智的脚前方网络

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:25:05 is just a simple little MLP so I want to start basically in a similar fashion also adding
只是一个简单的小MLP,所以我想基本上以类似的方式开始, 并且加上

1:25:11 computation into the network and this computation is on a per node level so I've already implemented
输入网络的计算, 而此计算是在每个节点水平上, 所以我已经实施了

1:25:18 it and you can see the diff highlighted on the left here when I've added or changed things
并且你可以看到左左边的 diff 突出显示 当我加或改变的东西

1:25:22 now before we had the multi-headed self- attention that did the communication
之前我们有多头的自我意识,

attention
注意力 注意力
神经网络基础

1:25:27 but we went way too fast to calculate the logits so the tokens looked at each other but
但是我们走得太快 计算日志 所以标语互相看对方,但...

logits
对数 分类模型生成的原始（非标准化）预测向量，通常会传递给标准化函数。如果模型要解决多 类别分类问题，则对数通常变成 softmax 函数 (https://www.tensorflow.org/api_docs/python/tf/nn/softmax_cross_entropy_with_logits_v2)的输入。 之后， softmax
神经网络基础

1:25:32 didn't really have a lot of time to think on what they found from the other tokens
并没有太多时间去思考 他们从其他的标志上找到了什么

1:25:37 and so what I've implemented here is a little feet forward single layer and this little layer is just
所以我在这里所执行的是 向前一层的一英尺, 这个小层是公正的

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:25:43 a linear filed by a relevant linearity and that's that's it so it's just a little layer and then I
由相关线性提交,所以它只是一层 然后

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:25:50 call it feet forward an embed and then this feet forward is just called sequentially right after
脚就叫它向前的嵌入 然后这脚就依次被命名

1:25:57 the self- attention so we self attend then we feet forward and you'll notice that the feet
自我注意,所以我们自顾自足 然后我们向前走 你会注意到脚

attention
注意力 注意力
神经网络基础

1:26:02 forward here when it's applying linear this is on a per token level all the tokens do this independently
当它应用线性时, 前面的此处, 此在每个标记级别上, 所有符号都独立进行此操作

1:26:08 so the self- attention is the communication and then once they've gathered all the data
所以自我注意就是交流 等他们收集了所有数据

attention
注意力 注意力
神经网络基础

1:26:13 now they need to think on that data individually and so that's what feet forward is doing
现在他们需要单独考虑数据 所以这就是前进的脚正在做的

1:26:17 and that's why I've added it here now when I train this the validation laws actually
所以我才在这里加上它 当我训练这个时 验证法实际上

validation
验证 验证
训练与优化

1:26:22 continues to go down now to 2.24 which is down from 2.28 the output still looked kind of
产出现在继续下降到2.24,从2.28下降到2.24, 产出看起来仍然有点像

1:26:28 terrible but at least we've improved the situation and so as a preview we're going to now start to
情况已经有所改善,所以作为预览 我们现在要开始

1:26:35 interspers the communication with the computation and that's also what the transformer does when it
与计算进行通信的连接, 变压器在它的时候也这样做

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:26:42 has blocks that communicate and then compute and it groups them and replicates them okay so let me
隔块有沟通和计算 并进行分组,复制它们

1:26:49 show you what we'd like to do we'd like to do something like this we have a block and this
展示我们想做什么 我们想做这样的事情 我们有一块块

1:26:53 block is basically this part here except for the cross attention now the block basically interspers
块块基本上就是此部分,除了交叉关注之外 现在块块基本上是中间线

attention
注意力 注意力
神经网络基础

1:26:59 communication and the computation the computation the communication is done using multi-headed
通信和计算信息通讯的计算使用多头

1:27:05 self- attention and then the computation is done using a feet forward network on all the tokens
自动自控, 然后用所有标记上的远方网络进行计算

attention
注意力 注意力
神经网络基础

1:27:10 independently now what I've added here also is you'll notice this takes the number of embeddings
现在我单独添加的也是 你也会注意到这需要嵌入数

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:27:18 in the embedding dimension and number of heads that we would like which is kind of like group
嵌入尺寸和头数的大小和数目, 我们希望这与群居相似

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

dimension
维度 维度
数学与微积分

1:27:21 sizing group convolution and I'm saying that number of heads we'd like is four and so because this is 32
规模化集团的混进,我说 我们想要的四位头 因为这32位

convolution [.kɒnvə'lu:ʃən]
卷积 简单来说，卷积在数学中指两个函数的组合。在机器学习中，卷积结合使用卷积过滤器和输 入矩阵来训练权重。 机器学习中的 " 卷积 " 一词通常是卷积运算 或卷积层 的简称。 如果没有卷积，机器学习算法就需要学习大张量中每个单元格各自的权重。例如，用 2K x 2K 图像训练的机器学习算法将被迫找出 400 万个单独的权重。而使用卷积，机器学习
神经网络基础

1:27:28 we calculate that because this 32 the number of heads should be four
我们计算出,因为这32个头数 应该是4个

1:27:33 the head size should be eight so that everything sort of works out channel wise
头的大小应该是八个 这样所有事情都能顺利解决

1:27:38 so this is how the transformer structures sort of the the sizes typically so the head size will
这就是变压器结构 如何与大小相似, 所以头部大小将会

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:27:44 become eight and then this is how we want to interspers them and then here I'm trying to
变成8 然后这就是我们想如何 相互插入他们,然后我在这里尝试

1:27:49 create blocks which is just a sequential application of block block block so that we're
创建块块, 以创建块块。 这只是块块块块的相继应用, 这样我们就可以

1:27:54 interspersing communication feet forward many many times and then finally we decode now I actually
多次相互穿透通讯的脚 向前许多次 然后最后我们解码了 现在,我实际上

1:28:00 tried to run this and the problem is this doesn't actually give a very good answer and very good
问题在于,这实际上并没有给出一个非常好的答案和很好的答复。

1:28:06 result and the reason for that is we're starting to actually get like a pretty deep neural net
结果和结果 原因就是 我们开始变得像一个非常深的神经网

neural net
神经网络 神经网络
神经网络基础

1:28:11 and deep neural nets suffer from optimization issues and I think that's what we're kind of
深厚的神经网因优化问题而受到影响 我想这就是我们

1:28:14 like slightly starting to run into so we need one more idea that we can borrow from the
有点像开始稍微开始进入 所以我们需要一个想法, 我们可以借到

1:28:20 transformer paper to resolve those difficulties now there are two optimizations that dramatically
解决这些困难的变压器纸 现在有两个优化

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:28:25 help with the depth of these networks and make sure that the networks remain optimizable let's
帮助这些网络的深度,确保网络保持最佳化。

1:28:30 talk about the first one the first one in this diagram is you see this arrow here and then this
谈到第一个这个, 这个图表的第一个, 就是您看到这箭头, 然后这个

1:28:36 arrow and this arrow those are skip connections or sometimes called residual connections they come
箭头和此箭头,这些是跳过连接或有时被称为剩余连接的箭头,它们来时是跳过链接或被称作剩余连接

1:28:41 from this paper the procedural learning from a direct mission from about 2015 that introduced the
本文件从本文件中总结了从2015年左右开始的直接任务中汲取的程序性经验,其中介绍了

1:28:48 concept now these are basically what it means is you transform the data but then you have a skip
现在这些概念的概念 基本上就是它的意思是 你转换数据,但你有一个跳过

1:28:55 connection with addition from the previous features now the way I like to visualize it
与前面的功能相加,现在我喜欢这样想象

1:29:01 that I prefer is the following here the computation happens from the top to bottom
我更喜欢在这里进行以下的计算 计算从上到下

1:29:07 and basically you have this residual pathway and you are free to fork off from the residual
基本上你拥有这条剩余通道 你可以自由叉离剩余

1:29:12 pathway perform some computation and then project back to the residual pathway via addition and so

路径执行某些计算,然后通过添加等方法投回剩余路径

1:29:18 you go from the the inputs to the targets only the plus and plus and plus and the reason this

进料到目标 只有增、增、加

1:29:25 is useful is because during back propagation remember from our micrograd video earlier

是因为在后期传播时, 请记住我们先前的微格片段

back propagation

反向传播 反向传播

神经网络基础

1:29:31 addition distributes gradients equally to both of its branches that that fed as the input and so

加号向作为输入量供进的两处平等分配梯度,以便

1:29:38 the supervision or the gradients from the loss basically hop through every addition node

和每个加接点的连接或损失的梯度

loss [lɒs]

损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M

训练与优化

1:29:44 all the way to the input and then also fork off into the residual blocks but basically

一路到输入 然后叉入残留区块 但基本上

1:29:51 you have this gradient superhighway that goes directly from the supervision all the way to

您有这条梯度高高速公路, 从监管到

gradient ['greidiənt]

梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。

反向传播与梯度

1:29:56 the input unimpeded and then these original blocks are usually initialized in the beginning

输入不阻碍,然后这些原始区块通常在起始时初始化

1:30:01 so they contribute very very little if anything to the residual pathway they they are initialized

所以它们的贡献很小 如果对它们的剩余路径有什么贡献的话 它们被启动

1:30:05 that way so in the beginning they are sort of almost kind of like not there but then during

开始的时候,他们几乎有点像... ...不在那里,然后在

1:30:10 the optimization they come online over time and they start to contribute but at least at the
时间一到,他们开始贡献 但至少

1:30:17 initialization you can go from directly supervision to the input gradient is unimpeded
您可以从直接监督到输入梯度的初始化, 输入梯度不受阻碍

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

1:30:21 and just flows and then the blocks over time kick in and so that dramatically helps with the
顺着流,然后随着时间的推移 各个街区开始移动,所以这非常有助于

1:30:27 optimization so let's implement this so coming back to our block here basically what we want to do is
最优化 所以让我们执行这个计划 这样回到我们街区 这里基本上我们想做的就是

1:30:33 we want to do x equals x plus self- attention and x equals x plus self-taught feed forward
我们希望做x等于x+自关注,x等于x加上自教前进

attention
注意力 注意力
神经网络基础

1:30:40 so this is x and then we fork off and do some communication and come back and we fork off
所以这是x 然后我们走掉 做一些沟通 然后回来,我们走掉

1:30:46 and we do some computation and come back so those are residual connections and then swinging back up
然后我们做一些计算,然后回来 这样这些是剩余连接 然后回回回

1:30:52 here we also have to introduce this projection so nn.linear and this is going to be from
还要在这里引入这个投影, 以如此的 nn.线性, 这是来自

1:31:01 after we concatenate this this is the size unimpeded so this is the output of the self- attention
等我们结合这个大小之后, 这是不受阻碍的尺寸, 所以这是自我注意的结果

attention
注意力 注意力
神经网络基础

1:31:06 itself but then we actually want the to apply the projection and that's the result
但后来我们实际上想要应用预测结果,这就是结果

1:31:13 so the projection is just a linear transformation of the outcome of this layer
所以预测只是这个层的结果的线性转换

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:31:18 so that's the projection back into the residual pathway and then here in a feed forward it's going
所以这就是投影回向剩余路径 然后在前进的反馈中

1:31:23 to be the same thing I could have a self-taught projection here as well but let me just simplify
做我也可以在这里自我教学的投影 但是让我简化一下

1:31:28 it and let me couple it inside the same sequential container and so this is the
并让我把它配成一个相继的容器, 所以这就是

1:31:35 projection layer going back into the residual pathway and so that's well that's it
投射层回回到剩余路径,所以这是很好,就是这样

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:31:42 so now we can train this so I implemented one more small change when you look into the
这样我们就可以训练这个,所以我做了一个小的改变 当你看

1:31:47 paper again you see that the dimensionality of input and output is 512 for them and they're
输入和输出的维度是512, 它们

1:31:52 saying that the inner layer here in the feed forward has dimensionality of 2048 so there's
说进料前方的内层 具有2048年的维度 所以

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:31:57 a multiplier of four and so the inner layer of the feed forward network should be multiplied by
a乘以4,因此,进料前端网络的内层应乘以

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:32:03 four in terms of channel sizes so I came here and I multiplied four times embed here for the
以频道大小的四倍计 我来到这里,我乘以四倍

1:32:08 feed forward and then from four times and embed coming back down to an embed when we go back to
向前反馈,然后从4次 并嵌入后返回到一个嵌入 当我们回到

1:32:13 the projection to the projection so adding a bit of computation here and growing that layer
投影的预测 如此增加一点点计算 在这里,并生长该层

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:32:18 that is in the residual block on the side of the residual pathway and then I trained this and we
是在剩余路径的边边上的残留区块 然后训练了这个,我们

1:32:24 actually get down all the way to 2.08 validation loss and we also see that the network is starting
实际一直到2.08的验证损失, 我们也看到网络正在启动

validation
验证 验证
训练与优化

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:32:29 to get big enough that our train loss is getting ahead of validation loss so we started to see
才能大到足以让我们的火车损失 超过验证损失, 所以我们开始看到

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

validation
验证 验证
训练与优化

1:32:33 like a little bit of overfitting and our generations here are still not amazing but at least you see
有点过头了 我们这几代人还不是很神奇 但至少你看到了

overfitting
过拟合 过拟合
训练与优化

1:32:42 that we can see like is here this now grief sank like this starts to almost look like English so
我们看到像在这里,现在悲伤沉沉... ...这就像看起来几乎像英语

1:32:49 yeah we're starting to really get there okay and the second innovation that is very helpful for
第二次创新非常有助于

1:32:53 optimizing very deep neural networks is right here so we have this addition now that's the
最优化的深层神经网络就在这里 所以我们有了这个添加

图形处理单元 · 随机失活 1:32:58

1:32:58 residual part but this norm is referring to something called layer norm so layer norm is
部分剩余部分,但本规范是指一种称为层规范的事物,因此层规范就是

norm
范数 范数
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元 ，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:33:03 implemented in PyTorch it's a paper that came out a while back here and layer norm is very very
皮托尔特的一篇文章 本文在这里发表过一段时间 层规范非常

pytorch
PyTorch PyTorch
代码与工程

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:33:11 similar to bash norm so remember back to our make more series part three we implemented bash
类似打碎规范的打碎规范, 所以记住回到我们制作更多的系列第三部分 我们实施打碎

norm
范数 范数
数学与微积分

1:33:16 normalization and bash normalization basically just made sure that across the bash dimension
和制片人 和制版的正常化,基本上 只是确保

normalization [.nɔ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

dimension
维度 维度
数学与微积分

1:33:24 any individual neuron had unit Gaussian distribution so it was zero mean and
任何单个神经元都有高斯分布单位 所以是零平均值

distribution
分布 分布
数学与微积分

1:33:31 unit standard deviation one standard deviation output so what I did here is I'm copy pasting
单位标准差 1 标准差输出, 所以我在这里做的是复制粘贴

1:33:37 the bash norm 1d that we developed in our make more series and see here we can initialize
制成更多系列的Bash标准1D, 我们可以在这里开始

norm
范数 范数
数学与微积分

1:33:43 for example this module and we can have a batch of 32 100 dimensional vectors feeding through
例如,本模块,我们可以通过此模块 获得32100维矢量的一组食物

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:33:48 the bash norm layer so what this does is it guarantees that when we look at just the
标准层,所以这能保证 当我们只看

norm
范数 范数
数学与微积分

layer ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:33:54 zero column it's a zero mean one standard deviation so it's normalizing every single
零列,是0平均一个标准差 所以每条都正常

1:34:01 column of this input now the rows are not going to be normalized by default because we're just
输入的列中,行不会默认地被常规化,因为我们只是

1:34:07 normalizing columns so let's not implement layer norm it's very complicated look we come here
平整列的正常化,所以我们不要执行层规范 我们来这里很复杂

layer ['leiə]
层 神经网络 中的一组神经元 ， 负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

norm
范数 范数
数学与微积分

1:34:14 we change this from zero to one so we don't normalize the columns we normalize the rows
我们把这个从零改成一 这样我们就不能把列的正常化

1:34:21 and now we've implemented layer norm so now the columns are not going to be normalized
现在我们已经实施了层规范,所以现在 列不会正常化

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

norm
范数 范数
数学与微积分

1:34:29 but the rows are going to be normalized for every individual example it's 100 dimensional
但每列将按每个单个示例的 普通化,它就是100维

1:34:34 vector is normalized in this way and because our computation now does not span across examples
以这种方式使矢量正常化,因为我们的计算现在没有跨越示例

vector
向量 向量
数学与微积分

1:34:41 we can delete all of this buffer stuff because we can always apply this operation
我们可以删除所有这些缓冲文件, 因为我们可以总是应用此操作

operation
运算 运算
神经网络基础

1:34:48 and don't need to maintain any running buffers so we don't need the buffers we don't
不需要维持任何缓冲带 所以我们不需要我们不需要的缓冲带

1:34:55 there's no distinction between training and test time
训练时间和测试时间没有区别

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:34:59 and we don't need these running buffers we do keep gamma and beta we don't need the momentum we
我们不需要这些缓冲带 我们保留伽玛和贝塔 我们不需要我们的势头

1:35:04 don't care if it's training or not and this is now a layer norm and it normalizes the rows
管它是不是训练,现在这是分层规范,让排的正常化

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:35:12 instead of the columns and this here is identical to basically this here so let's now implement
而不是列,这里和这里的栏目完全相同 所以现在让我们执行

1:35:20 layer norm in our transformer before I incorporate the layer norm I just wanted to note that as I said
在我纳入层规范之前,我只想指出,正如我所说

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

norm
范数 范数
数学与微积分

1:35:25 very few details about the transformer have changed in the last five years but this is actually
过去五年变压器的变化细节很少 但实际上

transformer [træns'fɔ:mə]
Transformer 中文为“变换器”，但不够达雅，日常还是称英文 Transformer 中文为“变换器”，但不够达雅，日常还是称英文
神经网络基础

1:35:29 something that slightly departs from the original paper you see that the add and norm is applied
与原始文件稍略不同的东西,你可以看到,添加和规范是适用的

norm
范数 范数
数学与微积分

1:35:34 after the transformation but now it is a bit more basically common to apply the layer norm
转换后, 但现在应用层规范比较常见

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

norm
范数 范数
数学与微积分

1:35:43 before the transformation so there's a reshuffling of the layer norms so this is called the pre- norm
在变换之前, 层规范会重新调整 所以这叫做前北纬

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

norm
范数 范数
数学与微积分

1:35:48 formulation and that's the one that we're going to implement as well so select deviation from
并且我们将要执行的也是这个设计, 所以选择偏离

1:35:51 the original paper basically we need to layer norms layer norm one is an end dot layer norm
原始文件,基本上我们需要将规范层的规范层 规范层一是最终的圆层规范

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

norm
范数 范数
数学与微积分

1:35:58 and we tell it how many what is the embedding dimension and we need the second layer norm
我们告诉它有多少个嵌入维度 需要第二层规范

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

dimension
维度 维度
数学与微积分

1:36:05 and then here the layer norms are applied immediately on x so self dot layer norm one
在此, 并在此在此, 层规范立即适用于 x 。 所以自 dot 层规范 1

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

norm
范数 范数
数学与微积分

1:36:10 applied on x and self dot layer norm two applied on x before it goes into self- attention and
在自我注意之前和在自动注意前,对x应用自点层规范2和自点层准则2

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

norm
范数 范数
数学与微积分

1:36:16 feed forward and the size of the layer norm here is an embed so 32 so when the layer norm
向前 ff 和层规范的大小 这里是一个嵌入, 所以在层规范

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

norm
范数 范数
数学与微积分

1:36:23 is normalizing our features it is the normalization here happens the mean and the variance are taken
是我们把特征正常化,这是这里的正常化 发生中值和差异被取走

normalization [.nɒ:məlai'zeiʃən]
标准化 将实际的值区间转换为标准的值区间（通常为 -1 到 +1 或 0 到 1 ）的过程。例如，假设某个 特征的自然区间是 800 到 6000 。通过减法和除法运算，您可以将这些值标准化为位于 -1 到 +1 区间内。 另请参阅缩放 。
数学与微积分

1:36:32 over 32 numbers so the batch and the time act as batch dimensions both of them so this is kind of
数量超过32个以上,因此批量和时间作为批量尺寸,两者都具有一定的分批尺寸。

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:36:38 like a per token transformation that just normalizes the features and makes them a unit mean
就像是每个象征的转换, 只是使功能正常化 并使它们成为单位平均值

1:36:46 unit Gaussian at initialization but of course because these layer norms inside it have these
egossian 单位 开始时的 Gaussian 单位,但当然是因为其内部这些层规范有

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:36:52 gamma and beta trainable parameters the layer normal eventually create outputs that might not be
平层常量最终产生输出,但可能不是

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:36:59 unit Gaussian but the optimization will determine that so for now this is the this is incorporating
单位高西亚,但优化将决定,因此,现在这是它正在纳入

1:37:06 the layer norms and let's train them up okay so I let it run and we see that we get down to 2.06
层规范,让我们训练他们好起来 所以我让它运行,我们可以看到 我们得到到2.06

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:37:11 which is better than the previous 2.08 so a slight improvement by adding the layer norms
与前2.08相比好一点,因此通过增加层规范略微改进

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:37:16 and I'd expect that they help even more if we had bigger and deeper network
并且我期望他们能帮上更多忙 如果我们有更深更大的网络

1:37:20 one more thing I forgot to add is that there should be a layer norm here also typically as at the end
我忘了加一个我忘记加的一件事是 这里应该有一个层规范 通常也是在结尾

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

norm
范数 范数
数学与微积分

1:37:26 of the transformer and right before the final linear layer that decodes into vocabulary so I added
变压器的变压器, 就在最后线性层之前 解码成词汇层 所以我加了

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:37:33 that as well so at this stage we actually have a pretty complete transformer according to the original
而在现阶段 我们实际上有一个非常完整的变压器 根据原始

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:37:38 paper and it's a decoder only transformer I'll I'll talk about that in a second but at this
纸上,它是一个解码器,唯一的变压器 我一会儿再谈这个

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:37:43 stage the major pieces are in place so we can try to scale this up and see how well we can
舞台的主要部分已经到位, 这样我们可以尝试扩大这个范围, 看看我们有多好

1:37:47 push this number now in order to scale out the model I had to perform some cosmetic changes here
to
推这个数字,以便扩大模型的缩放。 我不得不在这里做一些化妆性修改

1:37:53 make it nicer so I introduced this variable called n layer which just specifies how many layers of the
更漂亮,所以我引入了一个叫做 n层的变量, 它只指定了

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。 Layers API (tf.layers) 一种 T
神经网络基础

1:37:59 blocks we're going to have I create a bunch of blocks and we have a new variable number of heads
结块,我们将会有 我创造一堆块 我们有一个新的可变的 头数

1:38:03 as well I pulled out the layer norm here and so this is identical now one thing that I did briefly
我在这里拔出了层规范,所以现在我做了一件简单的事情。

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

norm
范数 范数
数学与微积分

1:38:10 change is I added a dropout so dropout is something that you can add right before the
更改是 我加了一个辍学者,所以退学者可以 添加到之前

dropout
随机失活 随机失活
训练与优化

1:38:16 residual connection back right before the connection back into the residual pathway
连接返回剩余路径前的剩余连接回接

1:38:21 so we can drop out that as the last layer here we can drop out here at the end of the
以便我们能够退出,作为最后一层 这里的最后一层 我们可以在这里退出

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

1:38:27 multi-headed distinction as well and we can also drop out here when we calculate the
多重取多头的区别也各不相同,我们计算出

1:38:34 basically affinities and after the softmax we can drop out some of those so we can
基本上亲和关系,在软式后 我们可以退学一些 这样我们就可以

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:38:38 randomly prevent some of the nodes from communicating and so dropout comes from this
随机防止某些节点进行通信, 因此退学来自此

dropout
随机失活 随机失活
训练与优化

1:38:44 paper from 2014 or so and basically it takes your neural net and it randomly every forward
基本上它需要你的神经网, 它随机地向前移动

neural net
神经网络 神经网络
神经网络基础

1:38:53 backward pass shuts off some subset of neurons so randomly drops them to zero and trains without them
后向通行证关闭了某些子神经元, 因此随机将其降为零, 并训练没有它们

backward pass
反向传播 反向传播
神经网络基础

1:39:02 and what this does effectively is because the mask of what's being dropped out has changed
这是因为被退学的面具已经改变

1:39:06 every single forward backward pass it ends up kind of training an ensemble of sub networks
最终训练了一组子网络

backward pass
反向传播 反向传播
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:39:13 and then at test time everything is fully enabled and kind of all of those sub networks are merged
然后在测试时,所有的一切 完全启用,并类型 所有子网络的合并

1:39:17 into a single ensemble if you can if you want to think about it that way so I would read the
合并成一个组合 如果你可以的话 如果你想这样考虑 这样我就能读

1:39:21 paper to get the full detail for now we're just going to stay on the level of this is a
来获得全部细节 现在,我们只是要保持

1:39:25 regularization technique and I added it because I'm about to scale up the model quite a bit
正规化技术,我加了它 因为我要提升模型的尺寸

1:39:30 and I was concerned about overfitting so now when we scroll up to the top we'll see that I
我担心现在的车体超时 翻到顶部的时候会看到

overfitting
过拟合 过拟合
训练与优化

1:39:36 changed a number of hyperparameters here about our neural net so I made the batch size be
改变了我们神经网的几部超参数 所以我做了批量尺寸

neural net
神经网络 神经网络
神经网络基础

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

1:39:40 much larger now 64 I changed the block size to be 256 so previously it was just eight
更大得多的64,我把区块大小改为256 所以以前只有8个

1:39:46 eight characters of context now it is 256 characters of context to predict the 257th
上下文的八个字符,现在预测第257个是256个上下文字符

1:39:54 I brought down the learning rate a little bit because the neural net is now much bigger so
我降低了一点学习率 因为神经网现在更大了

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。
学习速率是一个重要的超参数 。
训练与优化

neural net
神经网络 神经网络
神经网络基础

1:39:58 I brought down the learning rate the embedding dimension is now 384 and there are six heads
我降低了学习率 嵌入的维度现在是384 和有六头

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

embedding [em'bediŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

1:40:03 so 384 divide 6 means that every head is 64 dimensional as it as a standard
所以384分6 意味着每个头的体积是64维,作为标准

1:40:11 and then there are going to be six layers of that and the dropout will be a point two so every
然后有六层 6层的这个, 辍学者将是一个点 2 所以每

dropout
随机失活 随机失活
训练与优化

1:40:15 forward backward pass 20 percent of all of these intermediate calculations are disabled and dropped
20%的中间计算结果 被禁用和丢弃

backward pass
反向传播 反向传播
神经网络基础

1:40:22 to zero and then I already trained this and I ran it so drumroll how does it perform
零到零 然后我训练了这个 我跑得那么鼓鼓,怎么表现

1:40:29 so let me just scroll up here we get a validation loss of 1.48 which is actually quite a bit of an
所以让我在这里滚动一下 我们的确认损失1.48美元 这实际上有点

validation
验证 验证
训练与优化

loss [lɒs]
损失 一种衡量指标，用于衡量模型的预测 偏离其标签 的程度。或者更悲观地说 是衡量模型有多差。要确定此值，模型必须定义损失函数。例如，线性回归模型通常将均方 误差 用作损失函数，而逻辑回归模型则使用对数损失函数 。 M
训练与优化

1:40:37 improvement on what we had before which I think was 2.07 so we went from 2.07 all the way down to 1.48

改进我们之前的改进,我认为是2.07,所以我们从2.07一直下降到1.48

1:40:42 just by scaling up this neural net with the code that we have and this of course ran for a lot

只需通过扩大这个神经网 使用我们拥有的代码, 而这当然运行了很多

neural net
神经网络 神经网络
神经网络基础

1:40:47 longer this maybe trained for I want to say about 15 minutes on my a 100 GPU so that's a pretty

可能还要训练15分钟 说100GPU的15分钟

gpu
图形处理单元/图形处理器 图形处理单元/图形处理器
神经网络基础

1:40:53 good GPU and if you don't have a GPU you're not going to be able to reproduce this on a

良好的 GPU,如果你没有GPU,你就不能在

gpu
图形处理单元/图形处理器 图形处理单元/图形处理器
神经网络基础

1:40:57 CPU this would be I would not run this on a CPU or a MacBook or something like that you'll have to

CPU 这将是我不会在CPU或麦克布或者类似的东西上运行这个

cpu
中央处理器 中央处理器
神经网络基础

1:41:03 break down the number of layers and the embedding dimension and so on but in about 15 minutes we

分解层和嵌入维度的数量,等等。 但是在大约15分钟内我们

embedding [em'bedɪŋ]
嵌入 A learned, meaningful vector-valued representation of a discrete symbol such as a word, mapping it to a low-dimensional dense vector that captures useful features for a n
神经网络基础

dimension
维度 维度
数学与微积分

机器翻译 · 训练循环 1:41:09

1:41:09 can get this kind of a result and I'm printing some of the Shakespeare here but what I did also
能够得到这样的结果 我在这里印刷一些莎士比亚 但同样我做了

1:41:15 is I printed 10,000 characters so a lot more and I wrote them to a file and so here we see some
我打印了一万个字, 写了上一个文件,所以我们在这里看到一些

1:41:21 of the outputs so it's a lot more recognizable as the input text file so the input text file
输出的输出, 所以它比输入文本文件更容易识别。 因此输入文本文件

1:41:29 just for reference looked like this so there's always like someone speaking in this manner and
仅供参考 看起来像这样 所以总是有人说话的方式

1:41:36 our predictions now take on that form except of course they're they're nonsensical when you
我们的预测现在采取这种形式 除了当然,他们不敏感 当你

1:41:41 actually read them so it is every crimpty B house oh those prepation we give heed
读了它们,所以它就是每间B小屋, 我们所注意的那些预言

1:41:50 you know oh oh sent me you mighty lord anyway so you can read through this it's nonsensical of course
派你来给我,你这个大主... ...这样你就能读过它了 当然是没有意识的

1:42:03 but this is just a transformer trained on the character level for one million characters that
但这只是一个变压器, 训练了100万个字符的字符级

> **transformer** [træns'fɔ:mə]
> Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
> 神经网络基础

1:42:09 come from Shakespeare so they're sort of like blabber zone in Shakespeare like manner but it
莎士比亚的莎士比亞人,所以他们有点像 莎士 比亚的胡言乱语区一样有礼貌

1:42:14 doesn't of course make sense at this scale but I think I think still a pretty good demonstration
当然,在这种规模上没有意义 但我认为这还是个不错的示范

1:42:19 of what's possible so now I think that kind of like concludes the programming section of this video
可能发生的事情,现在我想... ...我认为这是种类似结束 这个视频的节目部分

1:42:28 we basically kind of did a pretty good job and of implementing this transformer but the picture
我们基本上做了相当不错的工作 并且实施这个变压器,但图画

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:42:34 doesn't exactly match up to what we've done so what's going on with all these digital parts here
和我们所做的并不完全吻合 所以这里的数码部件是怎么回事

1:42:39 so let me finish explaining this architecture and why it looks so funky basically what's
让我来解释一下这个建筑结构 以及它为什么看起来这么古怪,基本上什么

1:42:44 happening here is what we implemented here is a decoder only transformer so there's no component
发生在这里 我们在这里执行的是一个解码器 只有一个变压器,所以没有组件

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:42:50 here this part is called the encoder and there's no cross attention block here our block only has a
这里这个部分叫做编码器 这儿没有交叉注意区 我们的街区只有

attention
注意力 注意力
神经网络基础

1:42:57 self attention and the feet forward so it is missing this third in between piece here this piece
脚和脚都注意了 所以它失去了这块中间的第三个

attention
注意力 注意力
神经网络基础

1:43:03 does cross attention so we don't have it and we don't have the encoder we just have the
交叉关注,所以我们没有它 我们没有编码器 我们只有

attention
注意力 注意力
神经网络基础

1:43:07 decoder and the reason we have a decoder only uh is because we are just generating text and it's
解码器和我们只有一个解码器的原因 是因为我们只是生成文本

1:43:13 unconditioned on anything we're just we're just blabbering on according to a given data set
任何我们只是根据特定数据组 胡言乱语

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ，用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

1:43:18 what makes it a decoder is that we are using the triangular mask in our transformer so it has
是什么使它成为一个解码器? 因为我们在变压器中使用三角面罩,所以它有

transformer [træns'fɔ:mə]
Transformer 中文为“变换器”，但不够达雅，日常还是称英文 Transformer 中文为“变换器”，但不够达雅，日常还是称英文
神经网络基础

1:43:24 this autoregressive property where we can just go and sample from it so the fact that it's using
利用这个自动递减属性, 我们可以从中提取样本, 以便了解它使用的事实

sample
样本 样本
神经网络基础

1:43:29 the triangulate triangular mask to mask out the attention makes it a decoder and it can be used
三角三角面罩,以遮盖注意力,使其成为一个解码器,可以使用

attention
注意力 注意力
神经网络基础

1:43:35 for language modeling now the reason that the original paper had an encoder decoder architecture
用于现在的语言模型化的语言,原纸有编码器解码器结构的原因

1:43:40 is because it is a machine translation paper so it is concerned with a different setting in particular
因为它是机器翻译纸,所以它特别关心不同的环境

machine translation
机器翻译 The task of automatically transforming text (or speech) from one natural language into another.
神经网络基础

1:43:46 it expects some tokens that encode say for example french and then it is expected to decode
期望一些代号上标的符号 比如法语 然后它会解码

1:43:53 the translation in english so so you typically these here are special tokens so you are expected
英文翻译,所以你通常会用这些 特别的标志来表示期望

1:44:00 to read in this and condition on it and then you start off the generation with a special
以此读取,并以此为条件,然后用特殊语言开始一代人

1:44:05 token called start so this is a special new token um that you introduce and always place in the beginning
开始一个自定义的符号, 所以这是一个特殊的新的标志, 嗯您可以介绍并总是在开头放置

1:44:12 and then the network is expected to output neural networks are awesome and then a special end token
然后网络将产生神经网络 真棒,然后一个特殊的终端标记

1:44:18 to finish the generation so this part here will be decoded exactly as we had we've done it neural
完成这代人 使这部分的解码 和我们之前一样

1:44:25 networks are awesome will be identical to what we did but unlike what we did they want to
与我们所做的完全一样 但和我们所做的不同

1:44:31 condition the generation on some additional information and in that case this additional
生成某些额外信息的条件,在这种情况下,这一附加信息

1:44:36 information is the french sentence that they should be translating so what they do now
信息是法语句,他们应该翻译成现在的翻译

1:44:42 is they bring the encoder now the encoder reads this part here so we're only going to take the
编码器读了这部分 所以我们只拿

1:44:49 part of french and we're going to create tokens from it exactly as we've seen in our video
一部分法语,我们将从它创造标志品 就像我们在视频中看到的

1:44:54 and we're going to put a transformer on it but there's going to be no triangular mask and so
我们准备用变压器装上它 但是没有三角面罩

transformer [træns'fo:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:44:59 all the tokens are allowed to talk to each other as much as they want and they're just encoding
所有的标志物都允许彼此交谈 尽可能多的交流,他们只是编码

1:45:04 whatever's the content of this french sentence once they've encoded it they they basically come
不管这句法语句子的内容是什么 一旦他们编码了它 他们基本上就来了

1:45:11 out in the top here and then what happens here is in our decoder which does the language modeling
顶部的这里 然后这里发生的事情 就在我们的解码器里

1:45:18 there's an additional connection here to the outputs of the encoder and that is brought in
此处还有另一个连接 与编码器输出的输出连接, 并被引入

1:45:24 through cross attention so the queries are still generated from x but now the keys and the values
通过交叉关注,所以查询仍然来自x,但现在关键和值

attention
注意力 注意力
神经网络基础

1:45:30 are coming from the side the keys and the values are coming from the top generated by the nodes that
键和值来自节点产生的顶部

1:45:36 came outside of the decode the encoder and those tops the keys and the values there the top of it
代码编码器的解码器外出,这些编码器顶端是密钥和值,顶端是它的顶端

1:45:43 feed in on a side into every single block of the decoder and so that's why there's an
一边喂进一个侧面 进入解码器的每个街区 所以这就是为什么有一个

1:45:48 additional cross attention and really what it's doing is it's conditioning the decoding
额外的交叉关注,实际上它正在做的是 它在调节解码

attention
注意力 注意力
神经网络基础

1:45:53 not just on the past of this current decoding but also on having seen the full fully encoded french
不仅在目前解码的过去,而且因为看到全部编码的法语

1:46:03 prompt sort of and so it's an encoder decoder model which is why we have those two transformers
快速的,所以它是一个编码器解码器模型 所以我们才有这两个变压器

1:46:08 an additional block and so on so we did not do this because we have no we have nothing to encode
再加上一个额外的块,等等,所以我们没有这样做,因为我们没有没有我们没有编码的

1:46:13 there's no conditioning we just have a text file and we just want to imitate it and that's why
没有调节设备 我们只是有一个文本文件 我们只是想模仿它 这就是为什么

1:46:17 we are using a decoder only transformer exactly as done in GPT okay so now I wanted to do a very
我们使用一个解码器 唯一的变压器 和GPT里完全一样

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

1:46:24 brief walkthrough of nano GPT which you can find on my github and uh nano GPT is basically two files
短短的穿过纳米GPT, 你可以在我的刺囊上找到, 呃纳米GPPT基本上就是两个文件

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

1:46:30 of interest there's train.py and model.py train.py is all the boilerplate code for training the
火车和模型 火车是所有训练的锅炉板代码

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:46:36 network it is basically all the stuff that we had here it's the training loop it's just that
网络基本上就是我们这里所有的东西 训练循环,只是

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:46:42 it's a lot more complicated because we're saving and loading checkpoints and pre-trained weights
因为我们在节省和加装检查站 以及训练前的重量

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:46:46 and we are uh decaying the learning rate and compiling the model and using distributed training
我们正在降低学习率,编集模型,并使用分布式培训

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:46:51 across multiple nodes or GPUs so the training .py gets a little bit more hairy complicated there's
跨多个节点或 GPU ,这样训练。 py 会变得多一点毛, 复杂一些

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:46:57 more options etc but the model.py should look very very similar to what we've done here in fact the
更多的选项等, 但是模型. py 应该看起来非常相似 我们在这里所做的事实上

图形处理单元 · 批次大小 1:47:04

1:47:04 model is is almost identical so first here we have the causal self- attention block and all of
模型几乎完全相同 所以首先在这里 我们有一个因果自用块

attention
注意力 注意力
神经网络基础

1:47:11 this should look very very recognizable to you we're producing query-ski's values we're doing dot
对你来说这应该非常明显 我们制作的就是Court -ski的价值观 我们做的是点

1:47:16 products we're masking applying softmax optionally dropping out and here we are pooling the values
使用软最大分的 自动退出,我们用这些产品来掩盖

softmax
Softmax函数是将一个实数向量转换为一个概率分布。 A function used in a neural network output layer that converts a vector of real-valued scores into a normalized probability distribution over discrete classes.
数学与微积分

1:47:24 what is different here is that in rcode I have separated out the multi-headed attention into
这里的不同之处在于 在代码中,我把多头目的关注 分离出来

attention
注意力 注意力
神经网络基础

1:47:31 just a single individual head and then here I have multiple heads and I explicitly concatenate them
只有一个人头,然后我有一个多头 我明确地把他们结合

1:47:37 whereas here all of it is implemented in a batched manner inside a single causal self- attention
此处所有这一切都在单一因果自觉中分批实施

attention
注意力 注意力
神经网络基础

1:47:43 and so we don't just have a b and a t and a c dimension we also end up with a fourth dimension
所以我们不能只有一个B和T 和C维 我们最后也有一个第四维

dimension
维度 维度
数学与微积分

1:47:48 which is the heads and so it just gets a lot more sort of hairy because we have four dimensional
因为我们有四维的

1:47:53 array um tensors now but it is um equivalent mathematically so the exact same thing is happening
equat 数组 um 发回式

1:47:59 as what we have it's just it's a bit more efficient because all the heads are not treated as a
因为我们拥有的只是它 只是它效率提高了一点 因为所有的头都不被视为

1:48:04 batch dimension as well then we have the multilayer perceptron it's using the gellu nonlinearity
分批尺寸和分批尺寸 然后我们有了多层的过光度 它使用的是凝胶非线性

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

dimension
维度 维度
数学与微积分

1:48:10 which is defined here except instead of relu and this is done just because openly I used it and I
此处定义除了除罗鲁之外, 这样做是因为我公开使用它,我公开使用

1:48:15 want to be able to load their checkpoints the blocks of the transformer are identical
想要能够装载他们的检查站, 变压器的区块相同

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:48:20 they communicate in the compute phase as we saw and then the gpt will be identical we have the
和我们所看到的在计算阶段的交流 然后这个GPT会完全相同

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

1:48:25 positioning coatings token encodings the blocks the layer norm at the end the final linear layer
设置涂层符号编码最后线性层末层的层规范区块

layer ['leiə]
层 神经网络 中的一组神经元，负责处理一组输入特征，或一组神经元 的输出。 此外还指 TensorFlow 中的抽象层。层是 Python 函数，以张量 和配置选项作为输 入，然后生成其他张量作为输出。当必要的张量组合起来后，用户便可以通过模型函数 将结果转换为 Estimator 。Layers API (tf.layers) 一种 T
神经网络基础

norm
范数 范数
数学与微积分

1:48:32 and this should look all very recognizable and there's a bit more here because I'm loading
并且这个应该看起来非常容易辨认 这里还有一点 因为我正在装货

1:48:36 checkpoints and stuff like that I'm separating out the parameters into those that should be
诸如此类的设置点和类似设置点, 我将参数分离出来, 并区分成应该

1:48:41 weight decayed and those that shouldn't but the generate function should also be very very
重重量衰减,那些不应该 但产生函数的功能 也应该非常

weight [weit]
权重 线性模型中特征 的系数，或深度网络中的边。训练线性模型的目标是确定每个特征 的理想权重。如果权重为 0 ，则相应的特征对模型来说没有任何贡献。
神经网络基础

1:48:45 similar so a few details are different but you should definitely be able to look at this
类似的细节不同,但你应该可以看这个

1:48:51 file and be able to understand a lot of the pieces now so let's now bring things back to
并且能够理解很多的片段 所以让我们现在把事情带回来

1:48:55 chat gpt what would it look like if we wanted to train chat gpt ourselves and how does it relate
和它有什么关系?

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

1:49:00 to what we learned today well to train chat gpt there are roughly two stages first is the pre-training
训练我们学习到的今天 训练聊天技巧 第一阶段大约有两个阶段

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:49:06 stage and then the fine-tuning stage in the pre-training stage we are training on a large
培训前阶段的微调阶段,我们正在对

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:49:12 chunk of internet and just trying to get a first decoder only transformer to babble text so it's
试图获得第一个解码器 只有一个变压器 来调用文字 所以它

chunk [tʃʌŋk]
词块 词块
神经网络基础

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:49:18 very very similar to what we've done ourselves except we've done like a tiny little baby
和我们自己所做的非常相似 除了我们像小宝宝一样

1:49:24 pre-training step and so in our case this is how you print a number of parameters I printed it and
训练前步骤,所以在我们的情况中, 这就是你如何打印一些参数 我印刷了它,

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:49:32 it's about 10 million so this transformer that I created here to create little shakespeare
大约1000万 所以这个变压器 我在这里创建的, 来创造小摇椅

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:49:38 transformer was about 10 million parameters our data set is roughly one million characters so
变压器大约是1000万参数 我们的数据集大约是100万个字符 所以

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ，用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

1:49:45 roughly one million tokens but you have to remember that opening eyes is different vocabulary
大约一百万个象征性的,但你必须记住 睁开眼睛是不同的词汇

1:49:49 they're not on the character level they use these um subword chunks of words and so they have a
他们没有在性格级别 他们使用这些小字词的小字组

1:49:55 vocabulary of 50 000 roughly elements and so their sequences are a bit more condensed
大约50000个元素的词汇表, 所以它们的序列会更压缩一些

1:50:01 so our data set the shakespeare data set would be probably around 300 000 tokens in the open
所以对我们的数据集 摇摆波亚数据集 将大约在30万个符号 在开放

data set
数据集 一组样本 的集合。 Dataset API (tf.data) 一种高级别的 TensorFlow API ，用于读取数据并将其转换为机器学习算法所需的格式。 tf.data.Dataset 对象表示一系列元素，其中每个元素都包含一个或多个张量 。 tf.data.Iterator 对象可获取 Dataset 中的元素。 如需详细了解 D
神经网络基础

1:50:06 ai vocabulary roughly so we trained about 10 million parameter model and roughly 300 000
大约是词汇表,所以我们训练了约1000万个参数模型和30万个参数模式

parameter [pə'ræmitə]
参数 机器学习系统自行训练的模型的变量。例如，权重 就是一种参数，它们的值是机器 学习系统通过连续的训练迭代逐渐学习到的。与超参数 相对。
神经网络基础

1:50:13 tokens now when you go to the gpt 3 paper and you look at the transformers that they trained
当你去Gpt3纸上看他们训练的变压器时, 现在的标记

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

1:50:21 they trained a number of transformers of different sizes but the biggest transformer here has 175
他们训练了不同大小的变压器 但最大的变压器有175个

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:50:26 billion parameters so ours is again 10 million they used this number of layers in the
使用这些层的

1:50:32 transformer this is the n embed this is the number of heads and this is the head size
变压器 这是 n 嵌入的 n 这是头数, 这是头大小

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:50:39 and then this is the batch size so ours was 65 and the learning rate is similar now when they
并且这是批量大小, 所以我们的是65, 而现在的学习率相似 当他们

batch [bætʃ]
批次 模型训练 的一次迭代 （即一次梯度 更新）中使用的样本 集。 另请参阅批次大小 。
训练与优化

learning rate
学习速率 在训练模型时用于梯度下降的一个标量。在每次迭代期间，梯度下降法 都 会将学习速率与梯度相乘。得出的乘积称为梯度步长。学习速率是一个重要的超参数 。
训练与优化

1:50:46 train this transformer they trained on 300 billion tokens so again remember ours is about 300 000 so
训练了3千亿个象征性的电压 记住我们的电压大约30万

transformer [træns'fɔ:mə]
Transformer 中文为"变换器"，但不够达雅，日常还是称英文 Transformer 中文为"变换器"，但不够达雅，日常还是称英文
神经网络基础

1:50:53 this is about a million fold increase and this number would not be even that large by today's
这个数字在今天甚至不会是那么大

1:50:58 standards you'd be going up one trillion and above so they are training a significantly larger
标准会上升一万亿以上 所以他们训练的比这要大得多

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:51:05 model on a good chunk of the internet and that is the pre-training stage but otherwise these
互联网上相当大块的模型,这是培训前阶段,但除此之外

chunk [tʃʌŋk]
词块 词块
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:51:12 hyper parameters should be fairly recognizable to you and the architecture is actually like
超超高参数应该可以相当识别,而这个结构实际上就像

1:51:16 nearly identical to what we implemented ourselves but of course it's a massive infrastructure
几乎和我们自己实施的东西一样 但当然,它是一个庞大的基础设施

1:51:21 challenge to train this you're talking about typically thousands of gpu 's having to you
训练这个挑战,你说的是 通常成千上万的gpu对你有压力

gpu
图形处理单元/图形处理器 图形处理单元/图形处理器
神经网络基础

1:51:26 know talk to each other to train models of this size so that's just a pre-training stage
互相交流,互相沟通 训练如此大小的模型 这样就只是个训练前阶段

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

训练数据 · 微调 1:51:31

1:51:31 now after you complete the pre- training stage you don't get something that responds to your
训练前阶段完成后 你得不到什么回应

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:51:36 questions with answers and it's not helpful and etc you get a document completer right so it babbles
回答问题,没有帮助等等 你得到一个完整的文档,所以它编织

1:51:44 but it doesn't babble Shakespeare it babbles internet it will create arbitrary news articles
却不说莎士比亚, 网络上编造谣言, 会制造专横的新闻文章

1:51:48 and documents and it will try to complete documents because that's what it's trained
文件和文件,它会试图完成文件 因为那是它训练的

1:51:51 for it's trying to complete the sequence so when you give it a question it would just
因为它试图完成序列的顺序 所以当你提问的时候 它会只是

1:51:56 potentially just give you more questions it would follow with more questions it will
可能只是给你更多的问题 它会接着更多的问题

1:52:00 do whatever it looks like the some close document would do in the training data on the internet
做任何看起来像某份近文件的 互联网上的培训数据

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:52:06 and so who knows you're getting kind of like undefined behavior it might basically answer
谁知道你的行为 有点像没有定义的行为

1:52:10 with two questions with other questions it might ignore your question it might just try to complete
加上两个问题,还有其它问题 它可能会忽略你的问题 它可能只是试图完成

1:52:15 some news article it's totally undermined as we say so the second fine tuning stage is to
新闻报导,它被彻底破坏 正如我们所说的,所以第二个微调阶段是

fine tuning
微调 微调
神经网络基础

1:52:21 actually align it to be an assistant and this is the second stage and so this chat gpt
将它与助手对齐, 这是第二阶段, 所以这个聊天格子

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

1:52:28 blog post from opening i talks a little bit about how this stage is achieved we basically
博客文章从开业i 开始,

1:52:35 there's roughly three steps to the to this stage so what they do here is they start to
大约有三个步骤 到这个阶段,所以他们在这里做的是开始

1:52:39 collect training data that looks specifically like what an assistant would do so they are
收集培训数据,具体地看似助理将如何做

training ['treinin]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:52:44 documents that have the format where the question is on top and then an answer is below and they
文档中带有问题位于顶部,然后回答在下面的格式的文档,这些格式

1:52:48 have a large number of these but probably not on the order of the internet this is probably
可能是因为网络上没有这样的命令,

1:52:53 on the order of maybe thousands of examples and so they they then fine tune the model to basically
以大概几千个例子的顺序排列, 因此他们然后精细地将模型 基本地

1:53:01 only focus on documents that look like that and so you're starting to slowly align it so
仅仅关注那些看起来像那样的文件 所以你开始慢慢地对齐

1:53:06 it's going to expect a question at the top and it's going to expect to complete the answer
上面会有一个问题,它会期望完成答案

1:53:10 and these very very large models are very sample efficient during their fine tuning
这些非常大的模型在微调过程中 是非常有效的样本

sample
样本 样本
神经网络基础

fine tuning
微调 微调
神经网络基础

1:53:15 so this actually somehow works but that's just step one that's just fine tuning
所以这实际上有些管用,但这只是第一步 微调

fine tuning
微调 微调
神经网络基础

1:53:20 so then they actually have more steps where okay the second step is you let the model respond
之后他们实际上还有更多的步骤, 在那里,好的第二步是 让模型响应

1:53:25 and then different raiders look at the different responses and rank them for their preference
然后不同的突击队员 仔细看不同的反应 并按他们的优先程度排列

1:53:29 as to which one is better than the other they use that to train a reward model
究竟谁比谁优呢?他们用谁优于谁,以便训练一个奖赏模式。

1:53:33 so they can predict basically using a different network how much of any candidate response
以便他们可以预测基本上使用不同的网络 来预测候选人的回答有多少

1:53:40 would be desirable and then once they have a reward model they run ppo which is a form of
最好能有这种优待,一旦他们有了奖赏模式,他们就会运行Ppo(一种形式)

1:53:47 policy gradient reinforcement learning optimizer to fine tune this sampling policy
政策梯度强化政策学习优化优化,以微调这一抽样政策

gradient ['greidiənt]
梯度 偏导数 相对于所有自变量的向量。在机器学习中，梯度是模型函数偏导数 的向量。梯度指向最高速上升的方向。
反向传播与梯度

optimizer
优化器 梯度下降法 的一种具体实现。 TensorFlow 的优化器基类是 tf.train.Optimizer (https://www.tensorflow.org/api_docs/python/tf/train/Optimizer)。不同的优化器 可能会利用以下一个或多个概念来增强梯度下降法在指定 训练集 中的效果：
训练与优化

1:53:54 so that the answers that the GPT now generates are expected to score a high reward according to
以便根据《全球防止酷刑和酷刑法》,现在产生的答案能够预期获得高额奖励。

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

1:54:01 the reward model and so basically there's a whole aligning stage here or fine tuning stage
奖赏模式,所以基本上这里有一个整齐的阶段 或者微调阶段

fine tuning
微调 微调
神经网络基础

1:54:07 it's got multiple steps in between there as well and it takes the model from being a document
中间还有多个步骤 模型从文档中取出

1:54:12 completer to a question answerer and that's like a whole separate stage a lot of this data is not
回答答题的完整全度,这就像一个完整的单独阶段 很多数据都不是

1:54:19 available publicly it is enthroned to open AI and it's much harder to replicate this stage
公开开放人工智能是众所皆可的,

1:54:26 and so that's roughly what would give you a chat GPT and nano GPT focuses on the pretraining stage
大约就是这个可以让你 聊天的GPT和NN NAM GPT 集中在训练前阶段

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

1:54:32 okay and that's everything that I wanted to cover today so we trained to summarize a decoder only
这也是今天我想报道的一切 所以我们训练只总结一个解码器

1:54:38 transformer following this famous paper attention is all you need from 2017 and so that's basically a
跟踪这个著名的纸质关注器的变压器 从2017年到2017年,你只需要注意

transformer [træns'fɔ:mə]
Transformer 中文为“变换器”，但不够达雅，日常还是称英文 Transformer 中文为“变换器”，但不够达雅，日常还是称英文
神经网络基础

attention
注意力 注意力
神经网络基础

1:54:46 GPT we trained it on a tiny Shakespeare and got sensible results all of the training code is roughly
高尔夫球 我们用小莎士比亚训练它 结果很合理 所有的培训守则都大致

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

training ['treiniŋ]
训练 确定构成模型的理想参数 的过程。
神经网络基础

1:54:55 200 lines of code I will be releasing this code base so also it comes with all the
密码200行 我将发布这个代码基

1:55:02 Git log commits along the way as we built it up in addition to this code I'm going to release the
基特日志在我们建造它的过程中 承诺继续前进 除了这个代码之外

1:55:09 notebook of course the Google collab and I hope that gave you a sense for how you can train
当然是Google collab的笔记本,我希望这能让你有意识 学会如何训练

1:55:16 these models like say GPT 3 that will be architecturally basically identical to what we have
这些模型,比如说GPT 3, 在建筑上将基本和我们拥有的完全相同

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

1:55:21 but they are somewhere between 10 000 and 1 million times bigger depending on how you count
视你数的大小而定

1:55:26 and so that's all I have for now we did not talk about any of the fine tuning stages that
所以我现在只有这些了 我们没有谈论任何细调阶段

fine tuning
微调 微调
神经网络基础

1:55:32 typically go on top of this so if you're interested in something that's not just language modeling but
通常会以此为顶尖 如果你对某件事感兴趣的话 不只是语言模型,而是

1:55:37 you actually want to you know say perform tasks or you want them to be aligned in a specific way
您实际上想要知道您是否真的想要说要执行任务或者要以特定的方式对齐

1:55:42 or you want to detect sentiment or anything like that basically anytime you don't want something
或者你想侦测情绪之类的东西 基本上只要你不想得到什么

1:55:47 that's just a document completer you have to complete further stages of fine tuning which
这只是一个文件完整器,你必须完成 进一步细微调整的阶段

fine tuning
微调 微调
神经网络基础

1:55:52 we did not cover and that could be simple supervised fine tuning or it can be something
我们没有覆盖,那可以很简单 监督细微调整 或者它可能是某事

fine tuning
微调 微调
神经网络基础

1:55:57 more fancy like we see in chat GPT we actually train a reward model and then do runs of
更像我们在聊天GPT里看到的花式, 我们实际上训练了一个奖励模式 然后跑

gpt
生成式预训练Transformer模型 生成式预训练Transformer模型
神经网络基础

1:56:01 BPO to align it with respect to the reward model so there's a lot more that can be down on top of it
与奖励模式保持一致 BPO 这样就有更多的

1:56:07 I think for now we're starting to get to about two hours mark so I'm going to kind of finish here
我想我们现在开始 到达两个小时的距离 所以我要在这里完成

1:56:14 I hope you enjoyed the lecture and yeah go forth and transform see you later
我希望你喜欢上课,是啊 出去变换,待会见

点击任意句子从该处播放 · 空格 播放/暂停 · T 中文 · C 连播 · ↑ ↓ 上下句 · Esc 停止
字幕由 ASR 转录并经 AI 校正重译，术语释义仅供参考。